

Deep Learning

Diffusion Models and Video Generation

Tuesday, 11/4/25

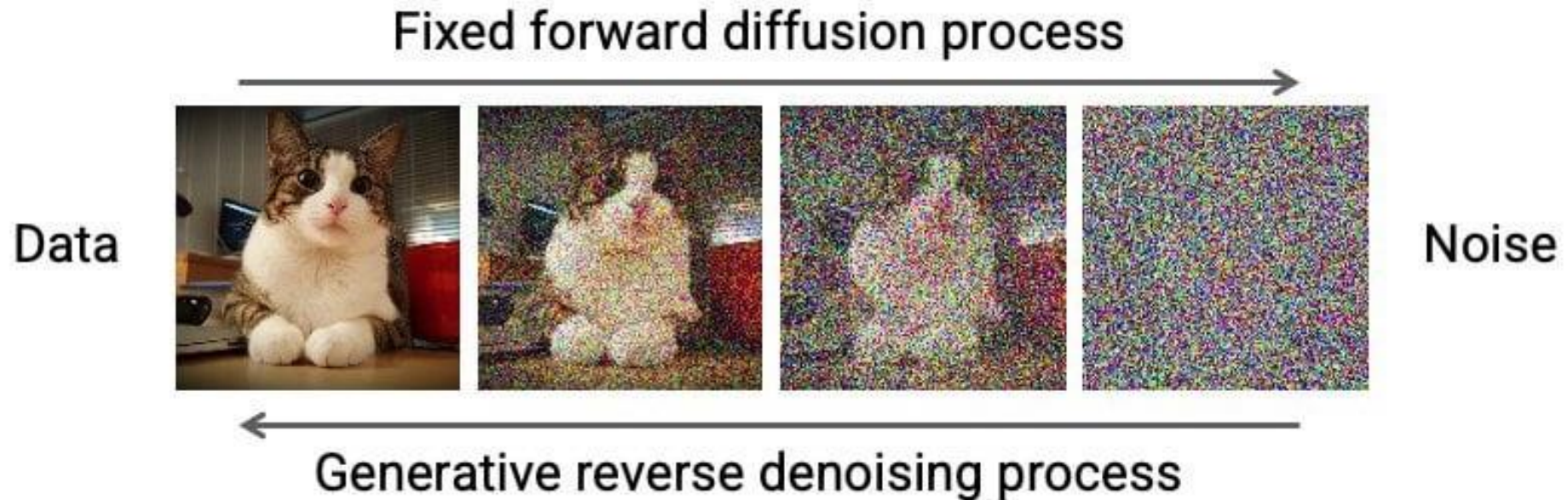
Eric Ewing



Denoising Auto Encoders

Denoising all the noise in one step may be too hard to learn.

What if we added and removed noise incrementally?

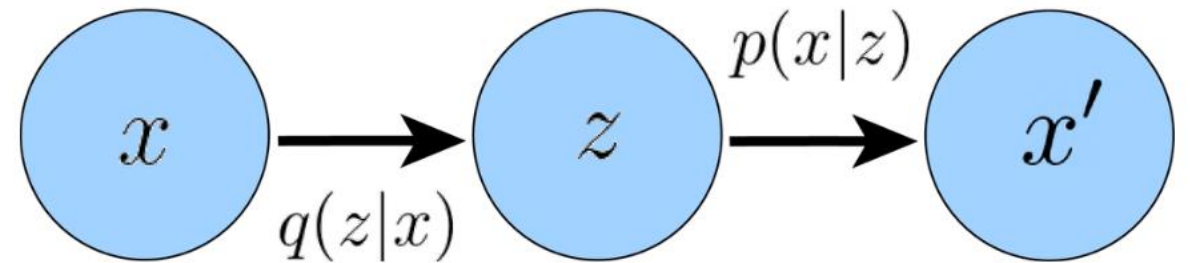
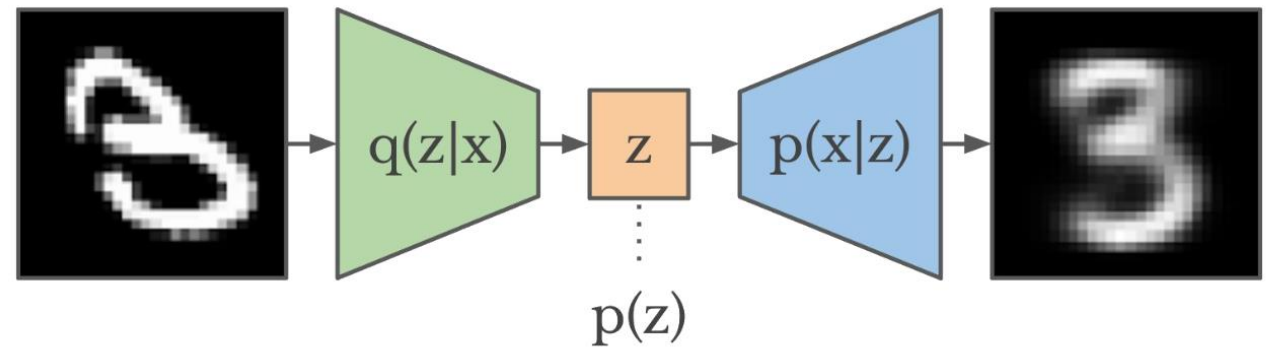


Variational Autoencoders

We can represent a VAE as a probabilistic graphical model

Encoder generates probability distribution $q(z|x)$

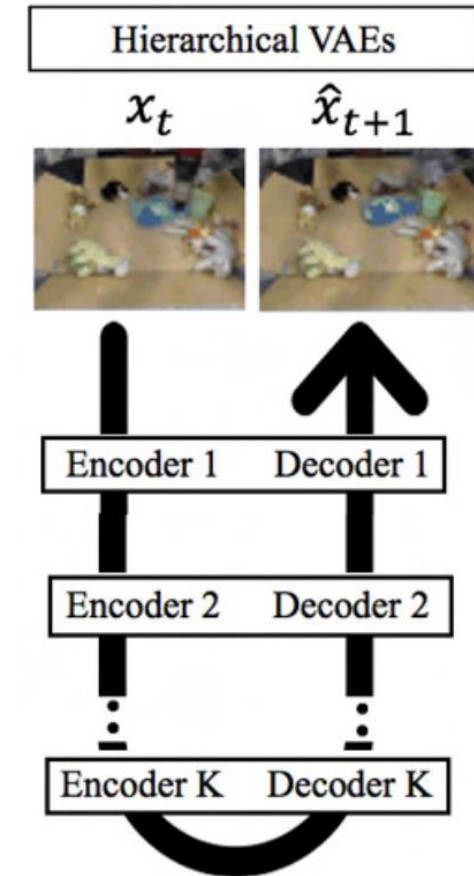
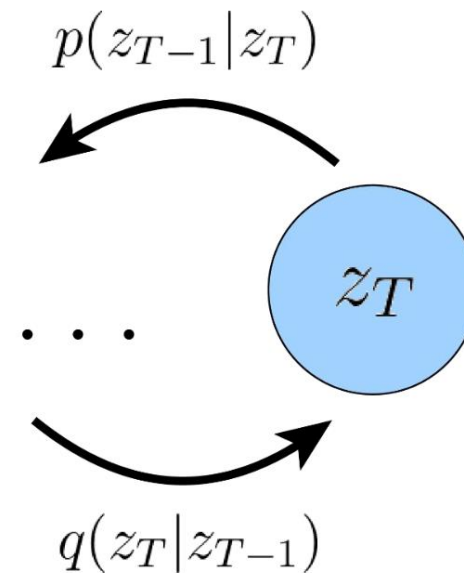
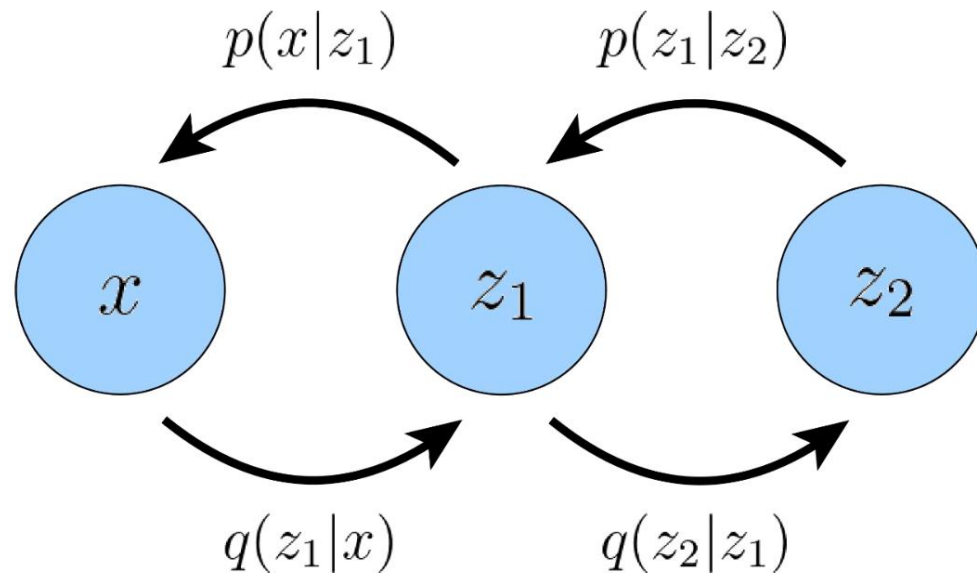
Decoder estimates $p(x|z)$



Hierarchical VAEs

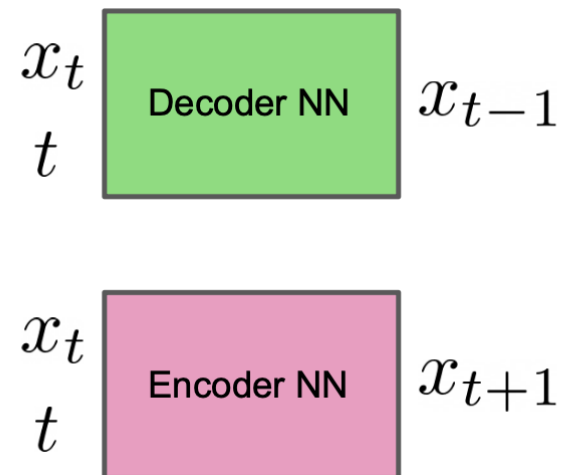
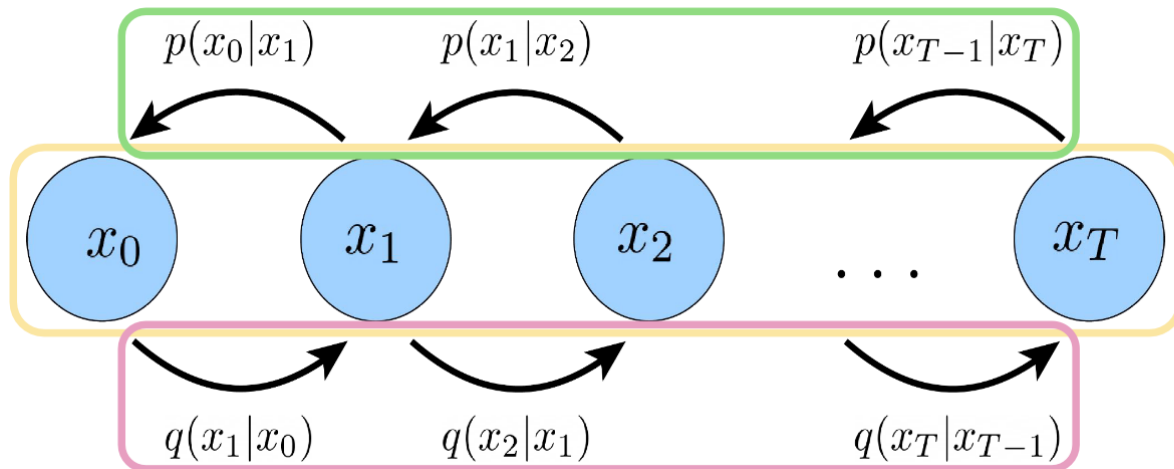
Idea: train K pairs of encoders and decoders

Each decoder is trained to reverse the associated encoder's operations



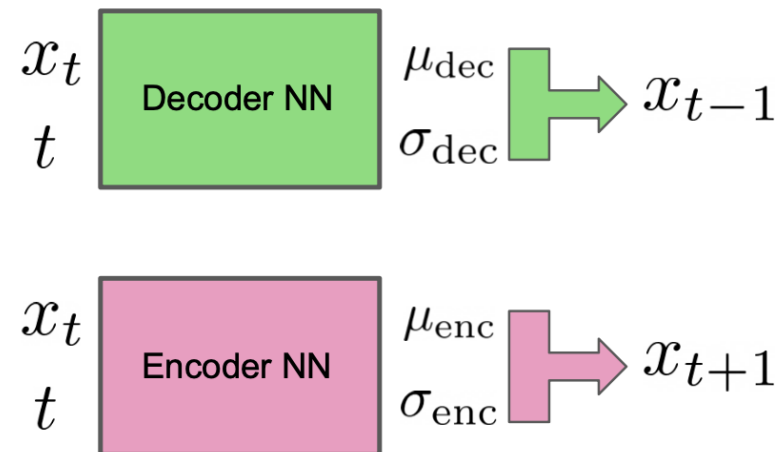
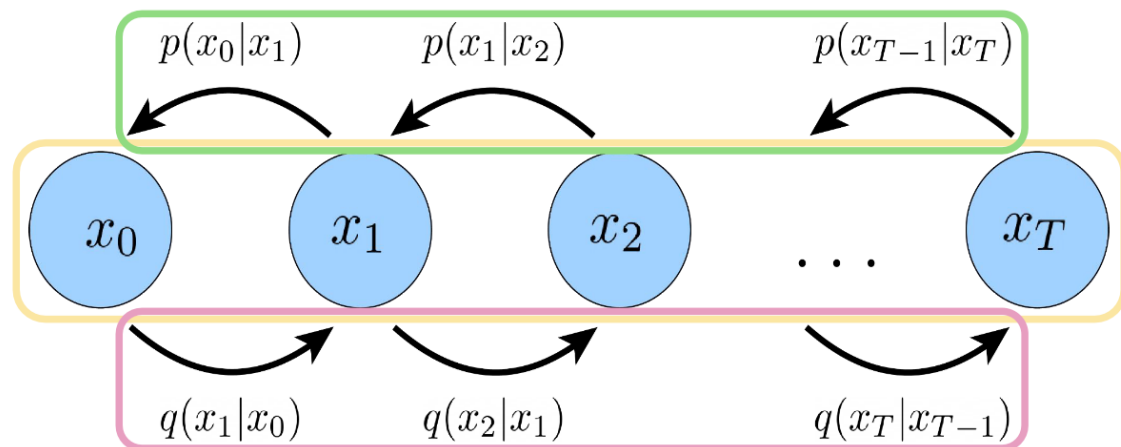
Hierarchical VAEs

A hierarchical VAE that keeps the dimensions the same can use the same decoder and encoder for all steps



Hierarchical VAEs

A hierarchical VAE that keeps the dimensions the same can use the same decoder and encoder for all steps



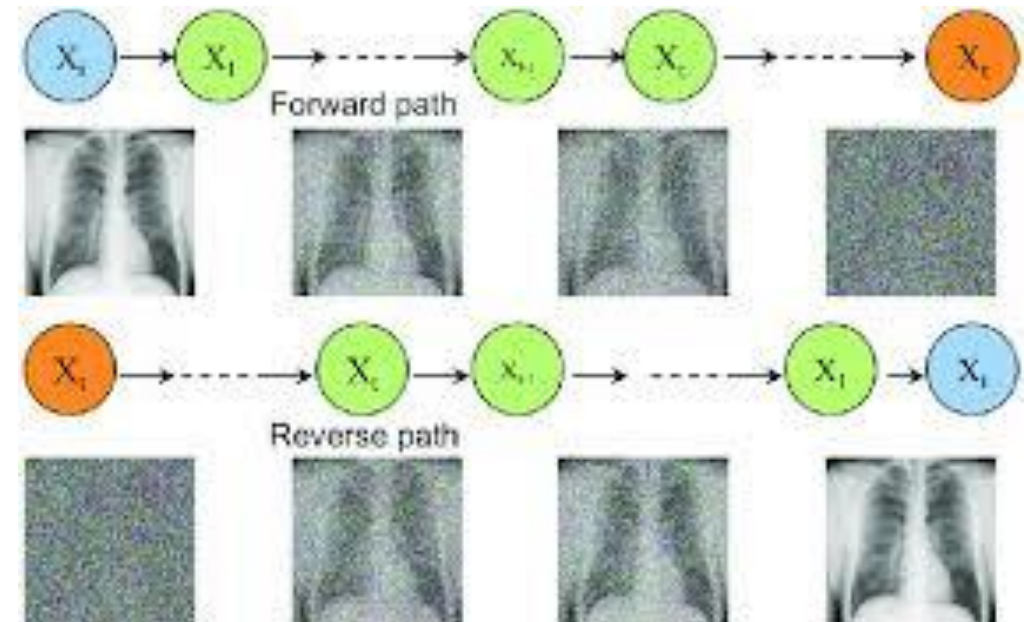
Denoising VAEs

If the size of the encoding in a VAE is the same size as the input, the model is no longer doing dimensionality reduction...

So why would we want this?

The forward process adds noise
The reverse path reverses this process

But we don't need to learn the encoders, adding noise isn't a learned process!

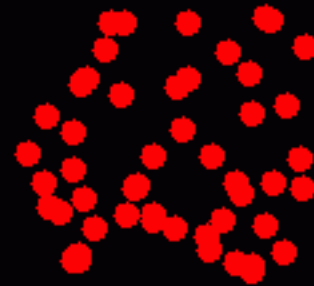


Why Call it Diffusion?

Diffusion of gas particles (and other physical things)

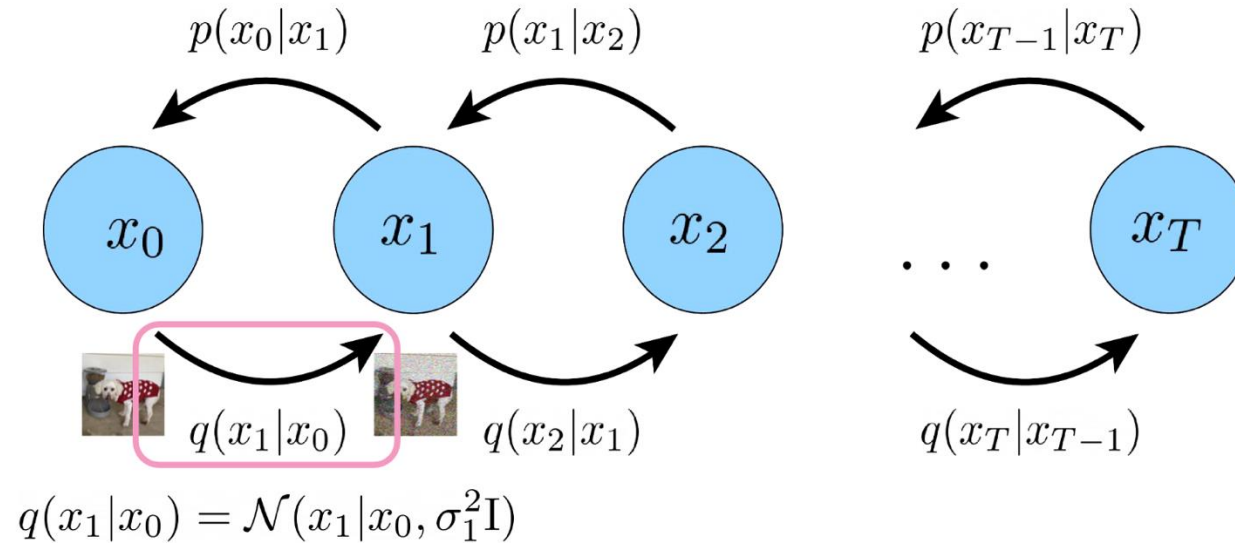
Start off organized

Transitions to “random noise”

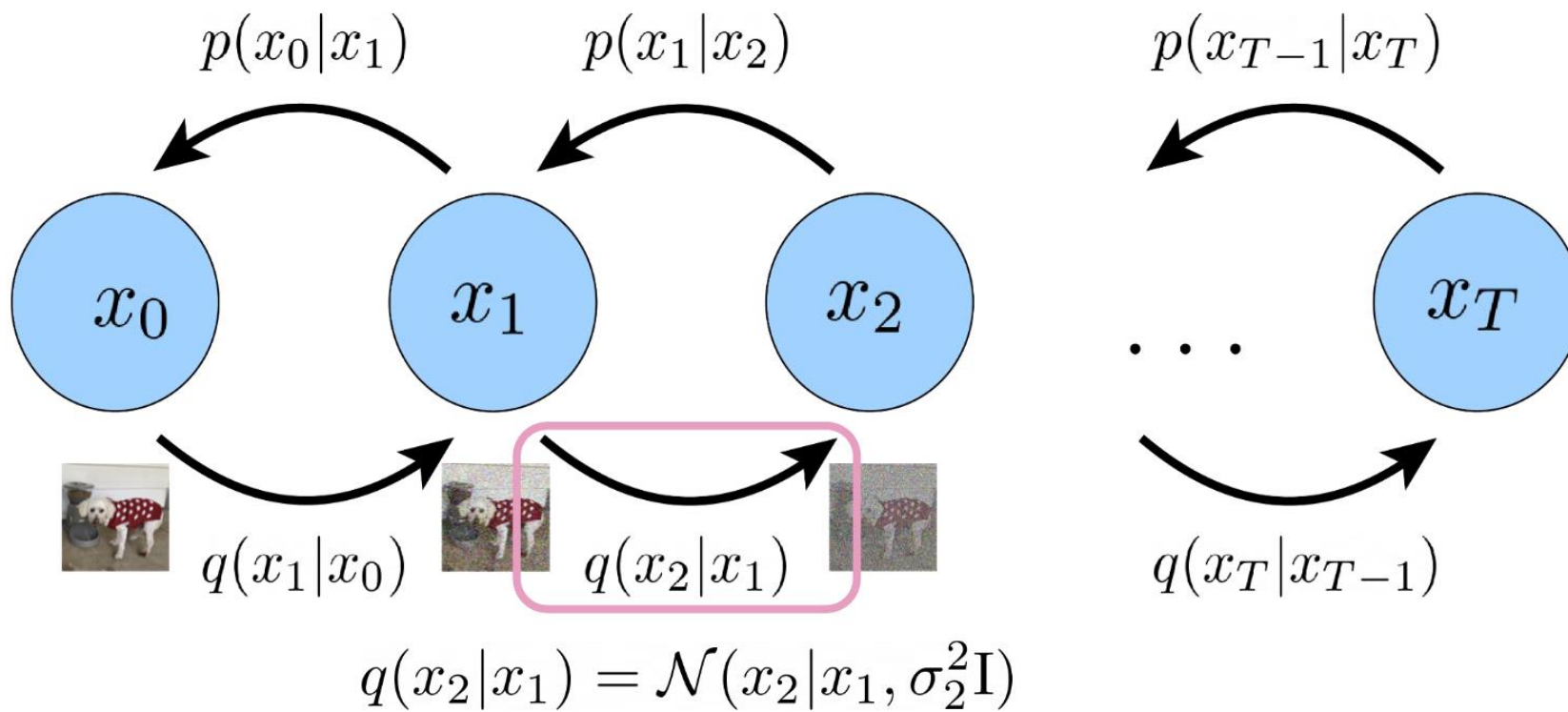


Adding Noise

What if each encoder transitions sample the input with some gaussian noise?



reparam. trick!

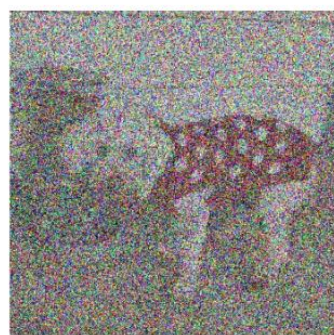
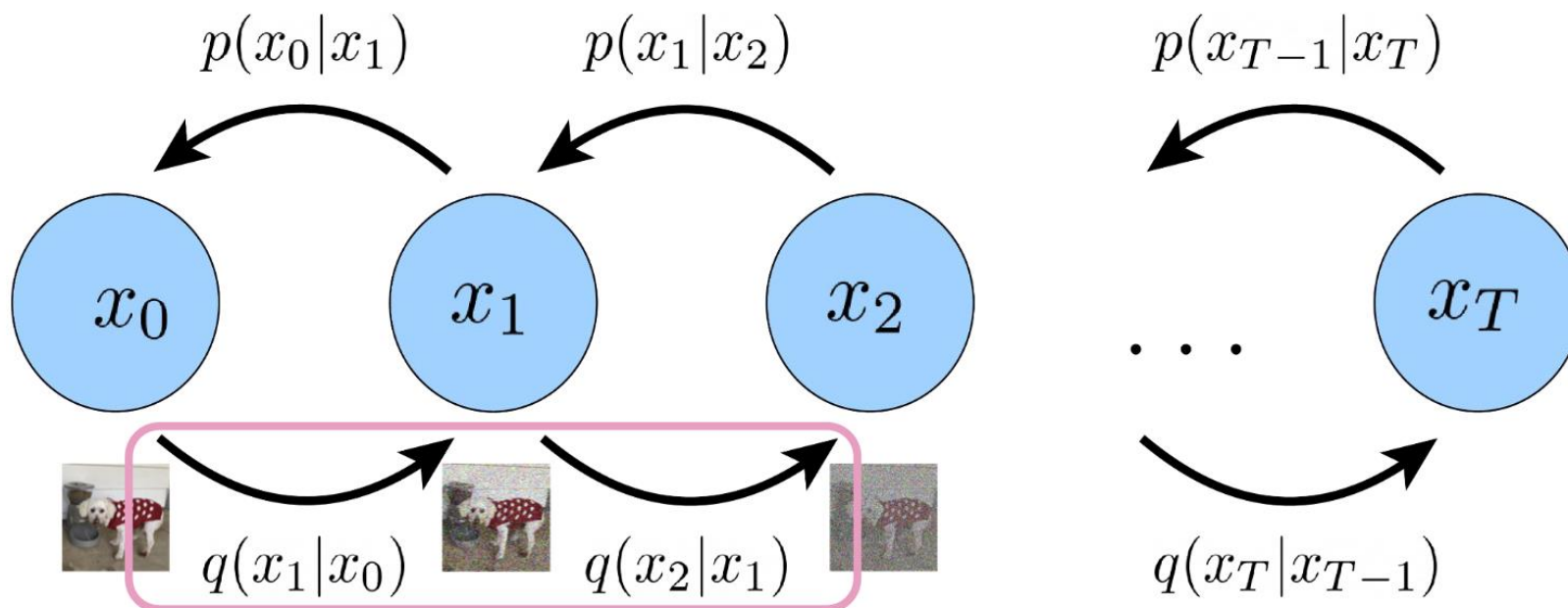


Visual representation of the reparameterization trick:

$$x_2 \sim q(x_2|x_1) = x_1 + \sigma_2 * \epsilon_0$$

Where $x_2 \sim q(x_2|x_1)$ is the noisy image, x_1 is the clear image, and ϵ_0 is the noise vector.

reparam. trick!



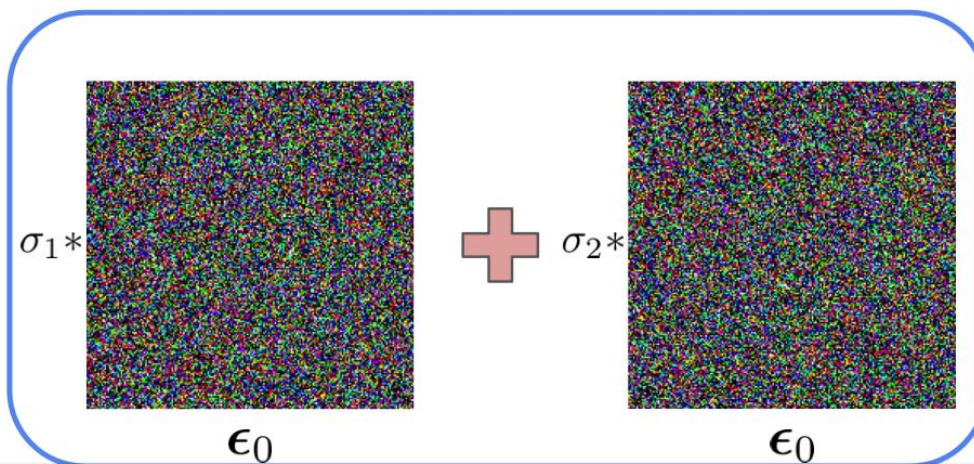
$x_2 \sim q(x_2|x_1)$

=



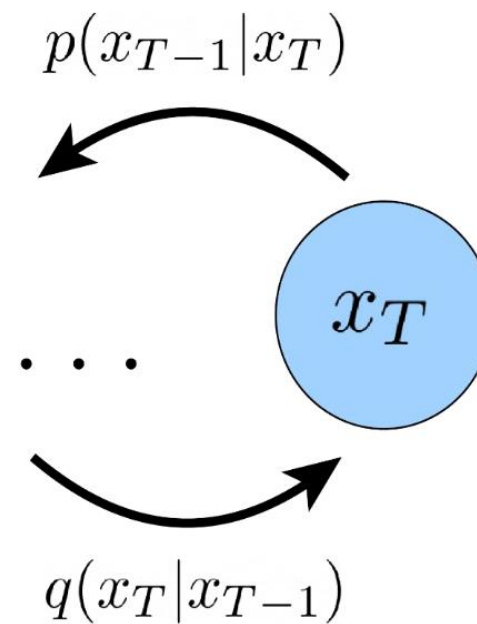
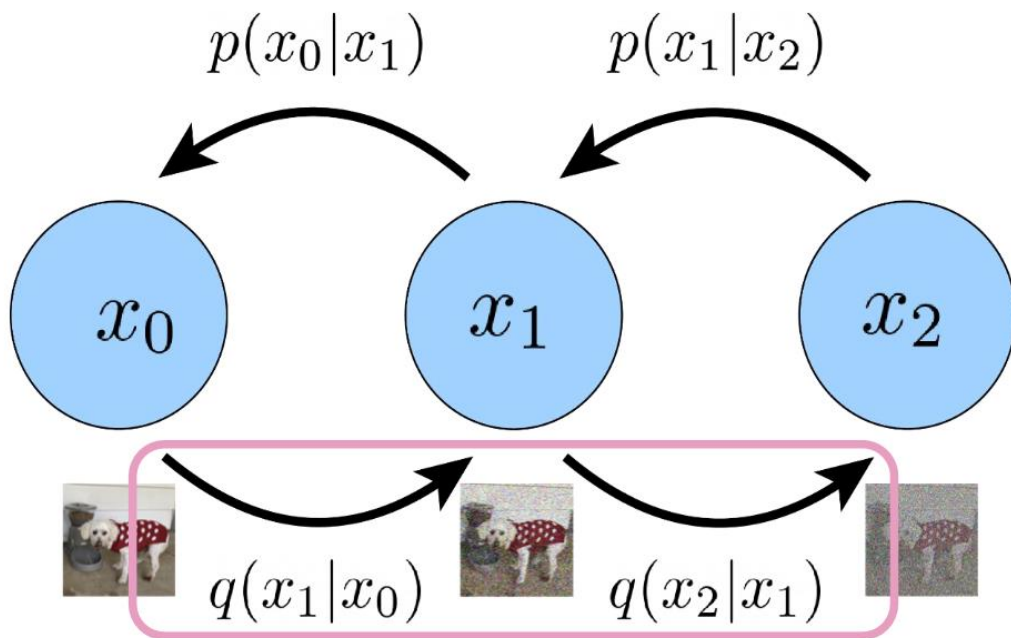
x_0

+



Aggregate into 1 sample!

reparam. trick!



where,

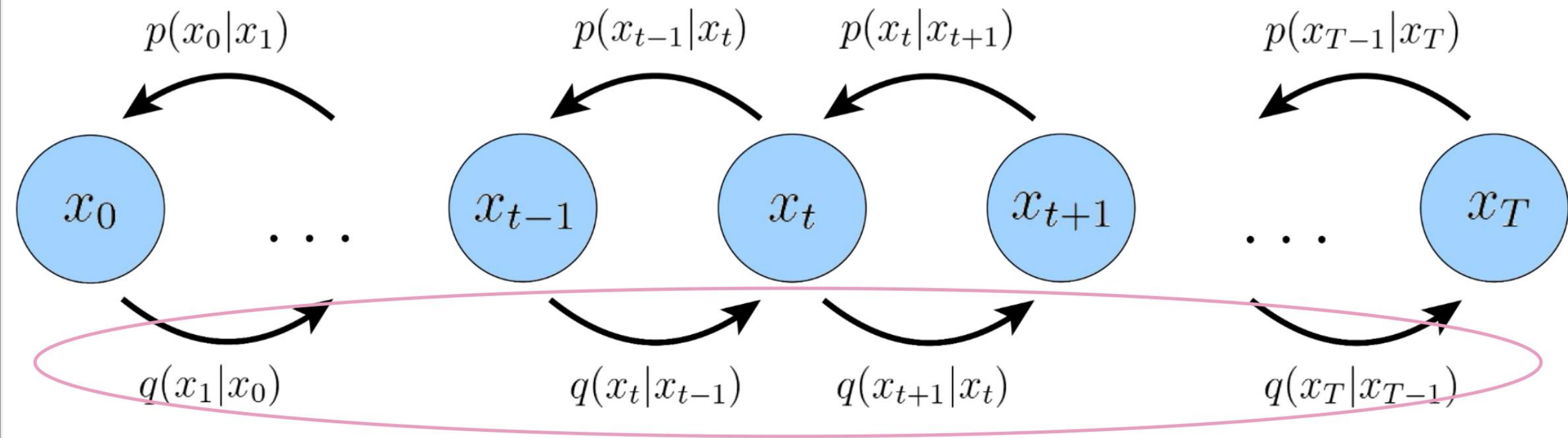
$$\alpha_2 = \sqrt{\sigma_1^2 + \sigma_2^2}$$

and,

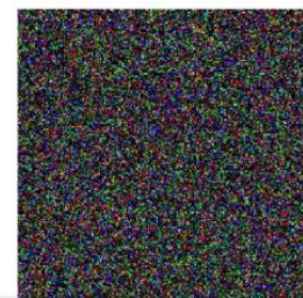
$$q(x_2|x_0) = \mathcal{N}(x_2|x_0, \alpha_2^2)$$

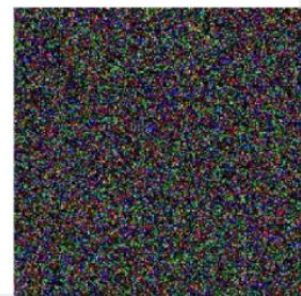
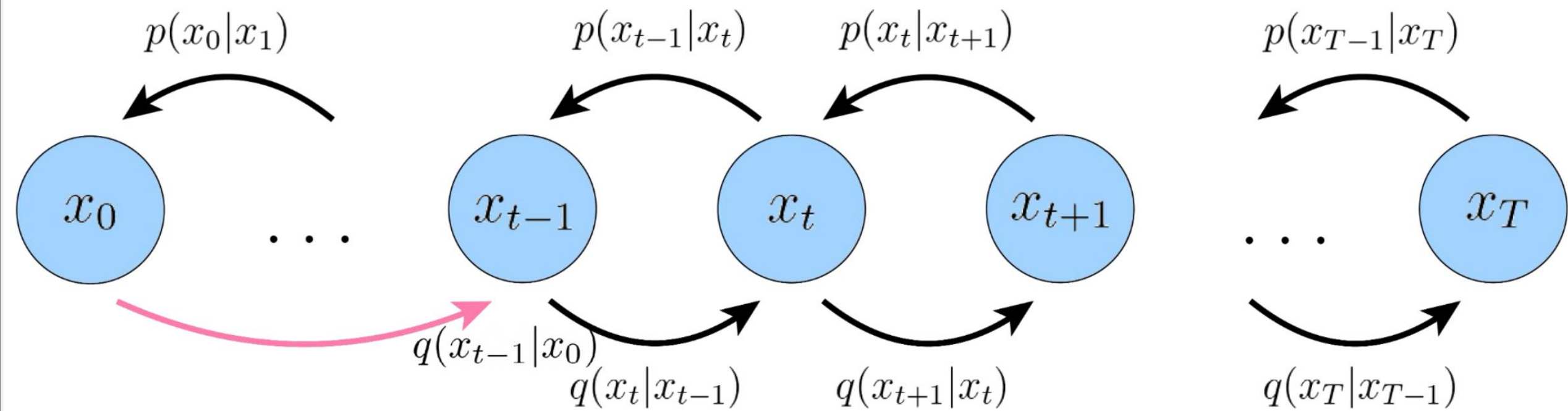
Aggregate into 1 sample!

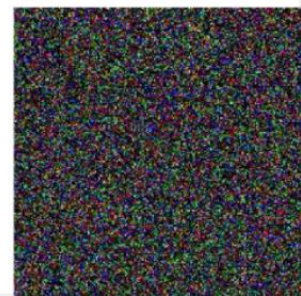
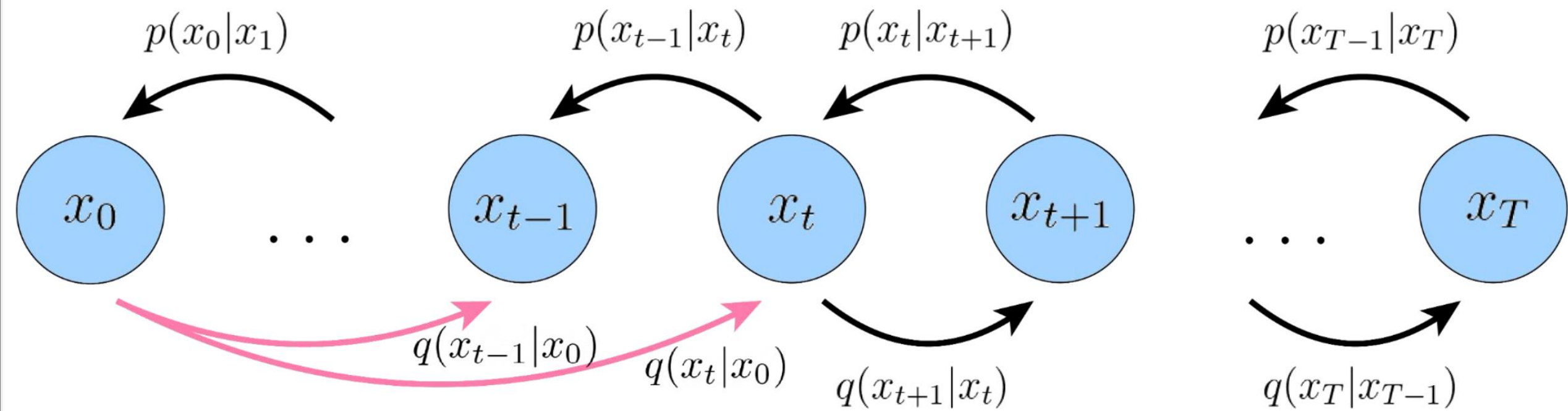
reparam. trick!



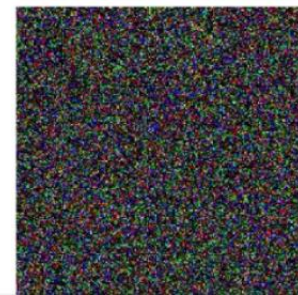
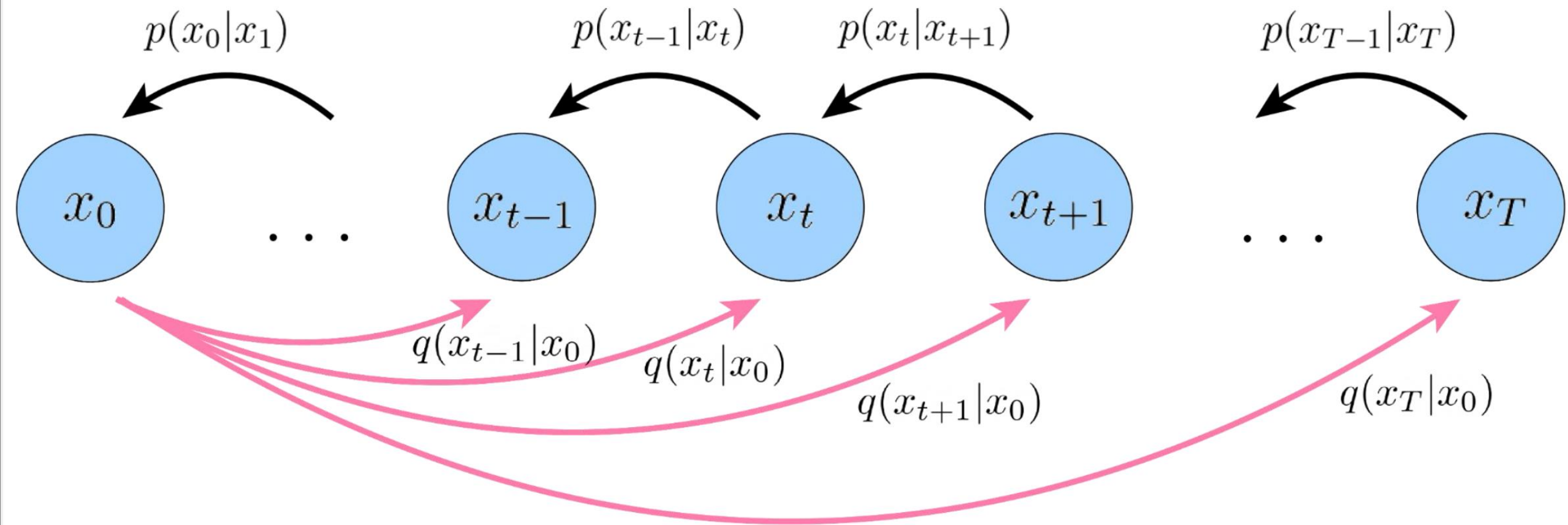
Individual (known) Gaussians!







$q(x_t|x_0)$ is a Gaussian, for arbitrary t !
 $q(x_t|x_0) = \mathcal{N}(x_t|x_0, \alpha_t^2\mathbf{I})$, where $\alpha_0, \alpha_1, \dots, \alpha_T$ are all known/fixed.



Forward Diffusion Process

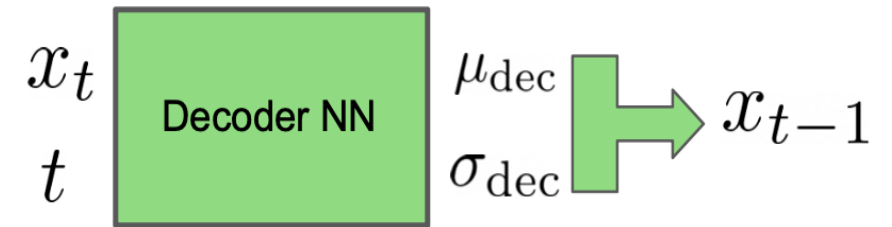
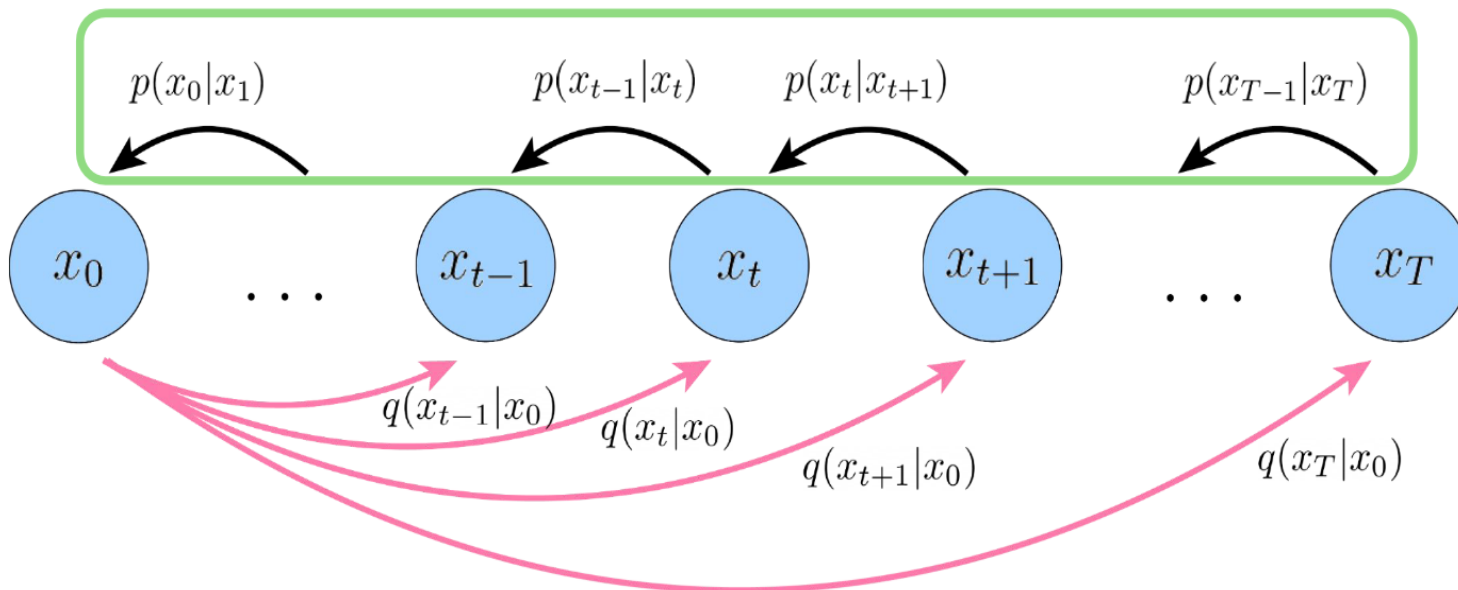
- The forward direction adds noise to the image
- We think of adding this noise incrementally, but we also have a way to sample from $q(x_t | x_0)$ directly!
 - No need to run a for loop when we can do it in one shot!

Diffusion Models

Are hierarchical VAEs with the following assumptions:

- All dimensions are the same (input size, encoding size)
- Encoder transitions are known gaussians centered around their previous inputs

Variations on vanilla diffusion models can remove these assumptions (i.e., make intermediate steps different shape

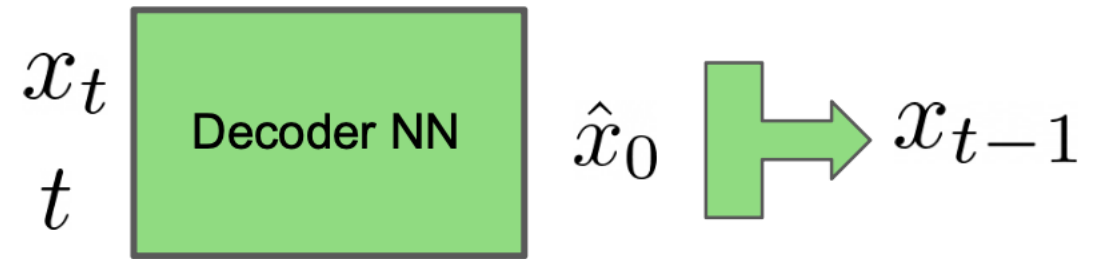


The Decoder

Diffusion models seek to learn a single neural network: a de-noising decoder

What form does x_{t-1} take?

$$q(x_{t-1}|x_0) = \mathcal{N}(x_{t-1}|x_0, \alpha_{t-1}^2 I)$$
$$x_t = x_0 + \alpha_{t-1} \epsilon$$



$$\hat{x}_{t-1} = \mu_{\text{dec}} + \sigma_{\text{dec}} * \epsilon$$

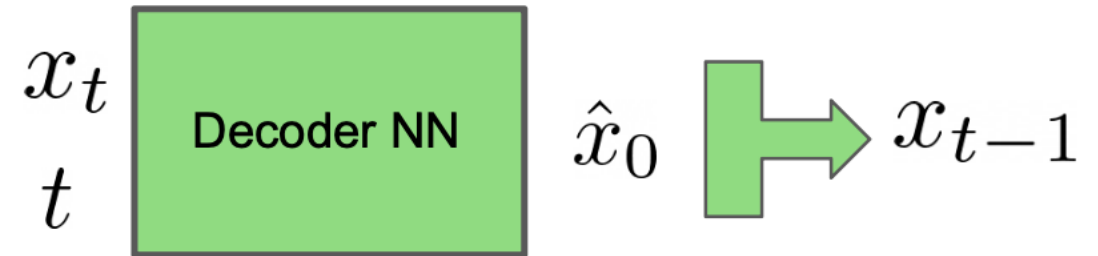
What is the ground truth for μ_{t-1} ?

Does the decoder even need to predict σ ?

Denoising Diffusion Models

Learn a single decoder network

- Input is image with added noise
- Output is predicted image with noise removed



How To Generate New Samples

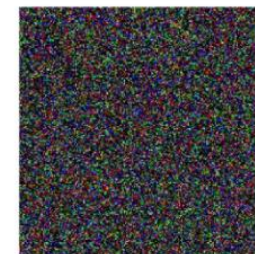
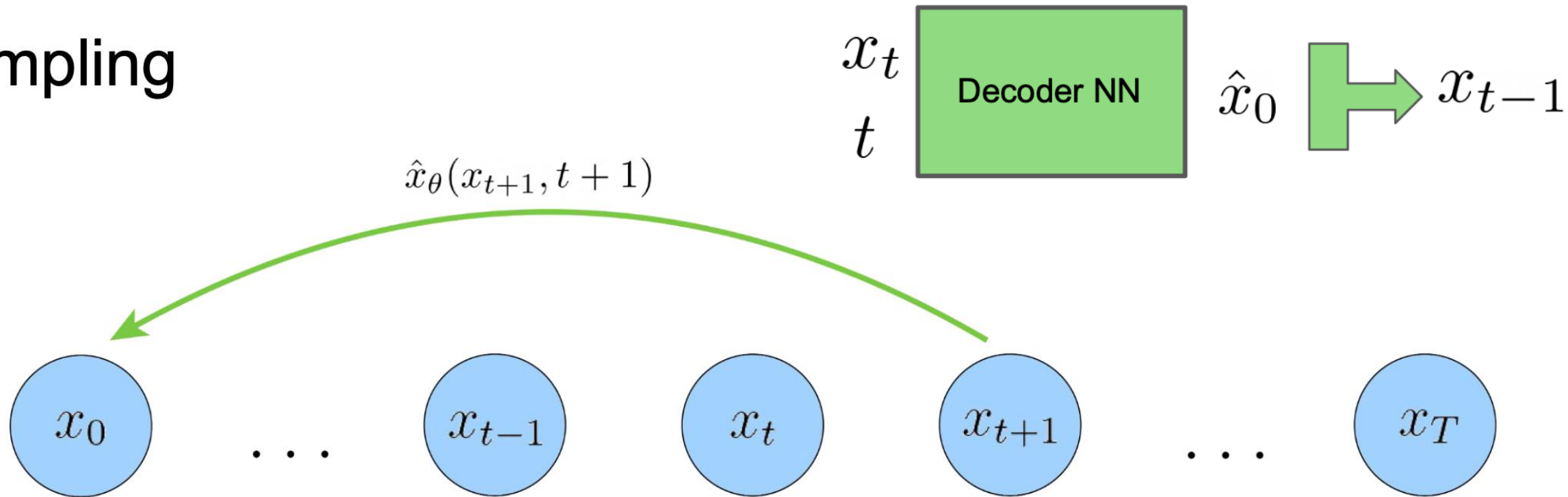
Idea 1: sample point from $\mathcal{N}(0, I)$, run decoder with $t=T$ to generate $\hat{x}_0$

Issue: Doesn't work that well... errors accumulate over time and become very hard to get right

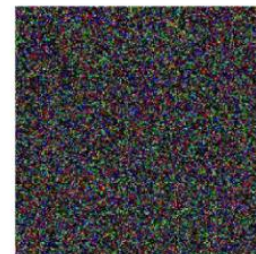
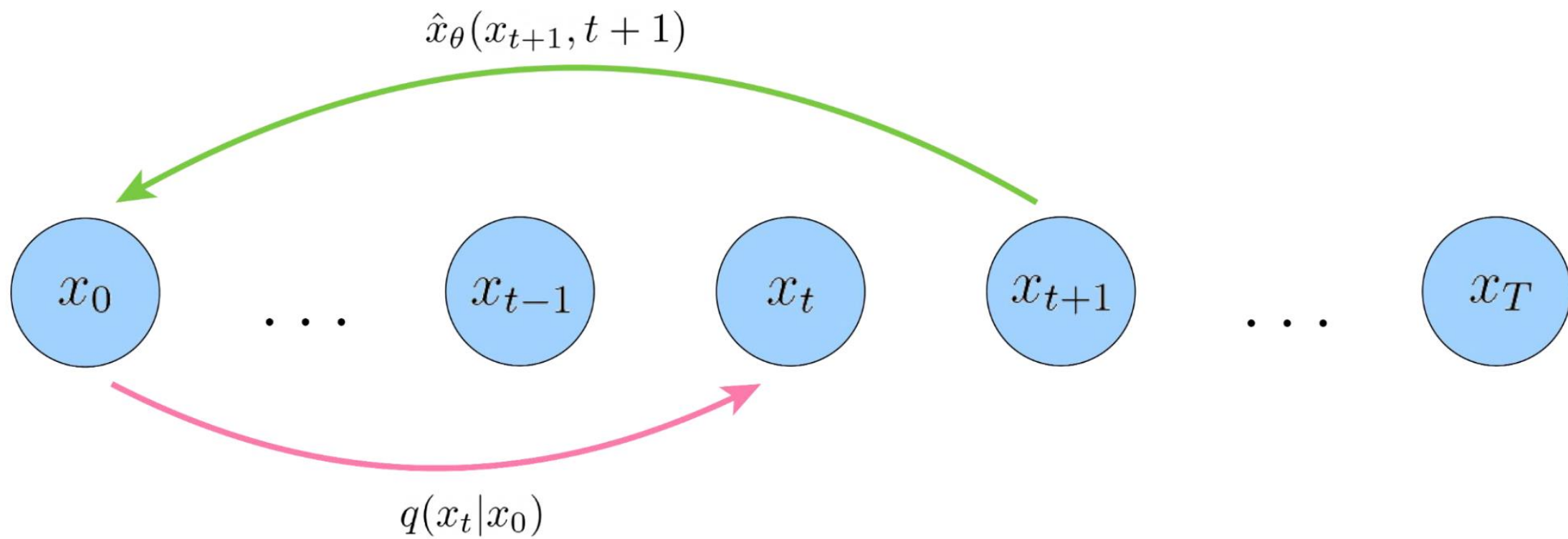
Idea 2: sample point from $\mathcal{N}(0, I)$, iteratively generate $\hat{x}_{t-1}$

But how do we actually generate $\hat{x}_{t-1}$ when our decoder generates $\hat{x}_0$?

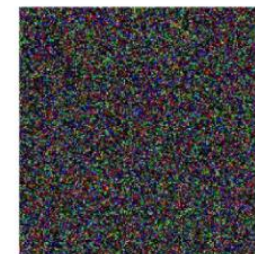
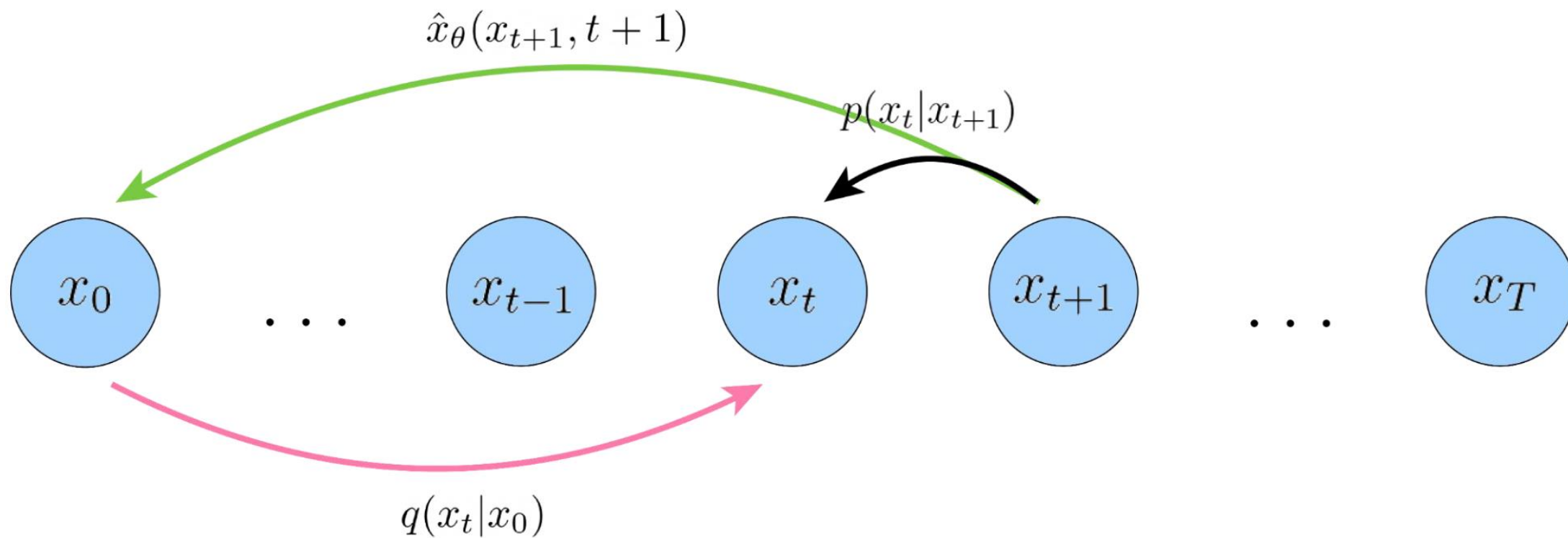
Sampling



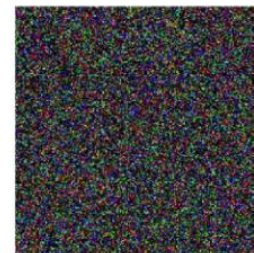
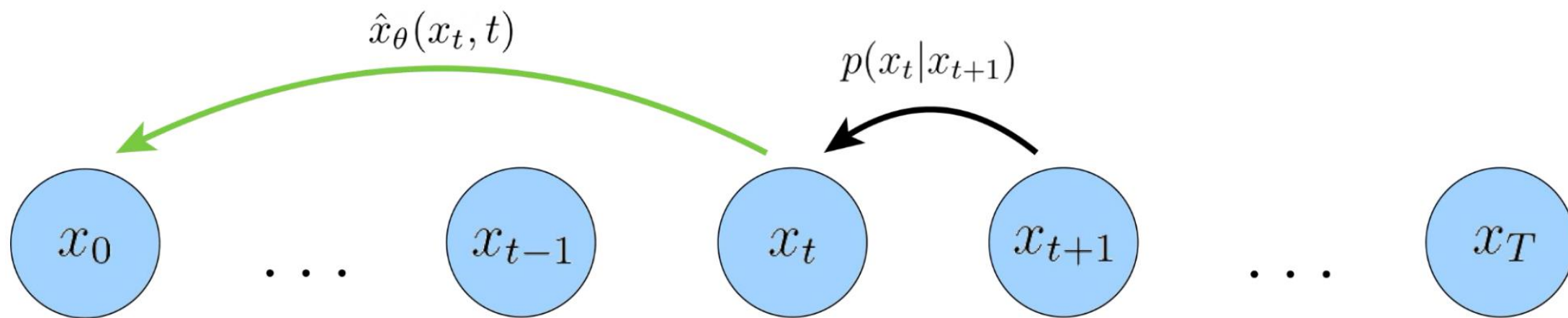
Sampling



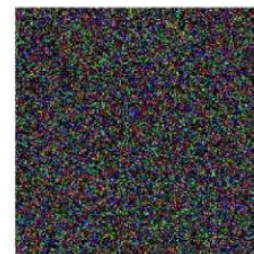
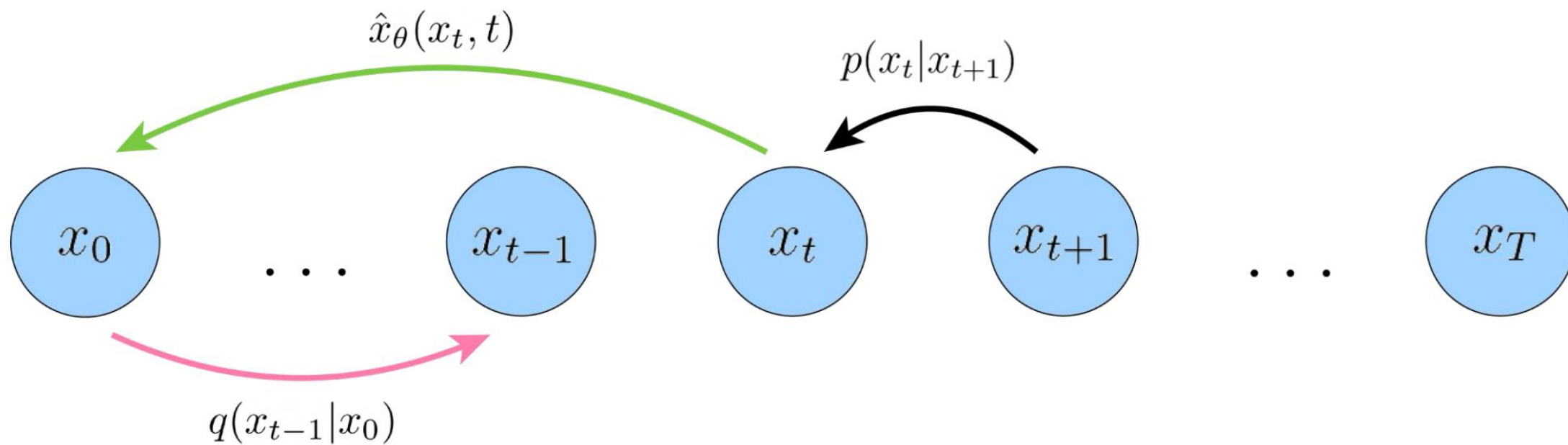
Sampling



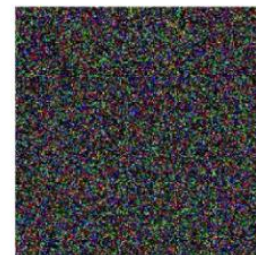
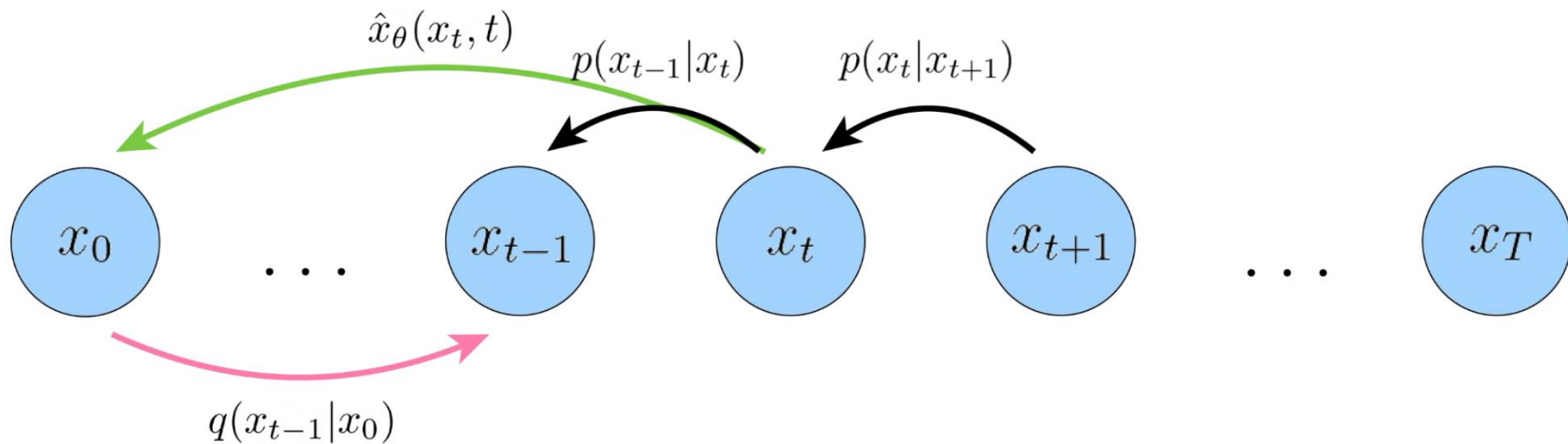
Sampling



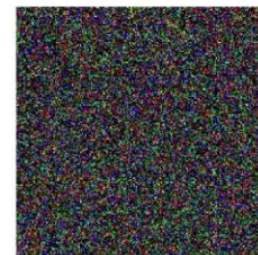
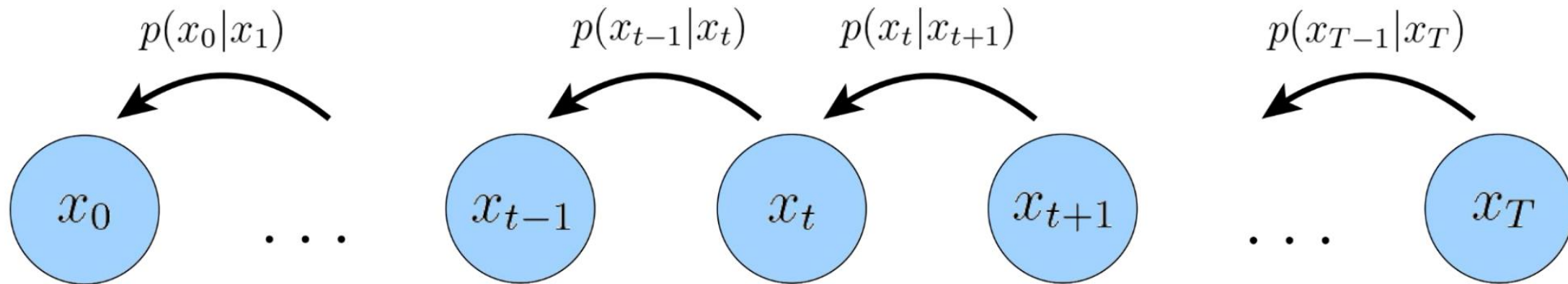
Sampling



Sampling



Sampling



Objective

There are many formulations of diffusion models:

1. Loss = Reconstruction Error + KL-Divergence to Prior (i.e., same as VAEs)
2. Simplified Objective: instead of predicting x_0 or x_t , predict ϵ , loss function is MSE between predicted ϵ and ground truth noise

Diffusion Training

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(1, \dots, T)$   
4:    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
6:      $\nabla_{\theta} ||\mathbf{x}_0 - \hat{\mathbf{x}}_{\theta}(\mathbf{x}_0 + \alpha_t \boldsymbol{\epsilon}, t)||^2$   
7: until converged
```

Algorithm 2 Sampling

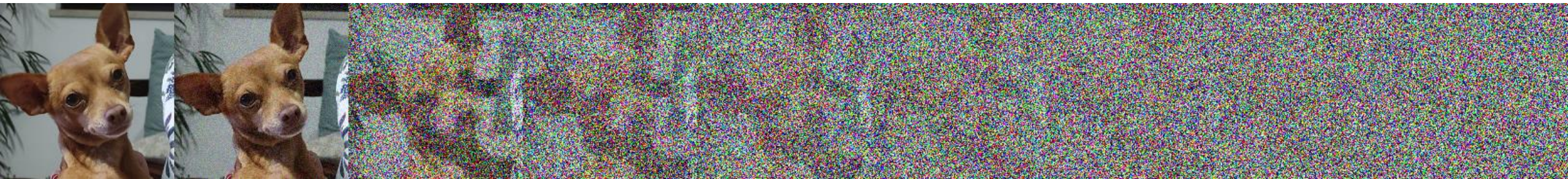
```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$ :  
3:    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\boldsymbol{\epsilon} = 0$   
4:    $\mathbf{x}_{t-1} = \hat{\mathbf{x}}_{\theta}(\mathbf{x}_t, t) + \alpha_{t-1} \boldsymbol{\epsilon}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

Noise Schedules

- What should the value of T be?
- How many steps of forward/reverse processes should we run?
- How much noise should be added at each step?

Amount of noise and number of steps determined by a *noise schedule* (hyperparameter)

Linear Schedule (equal noise added at each timestep)



Noise Schedules

- What should the value of T be?
- How many steps of forward/reverse processes should we run?
- How much noise should be added at each step?

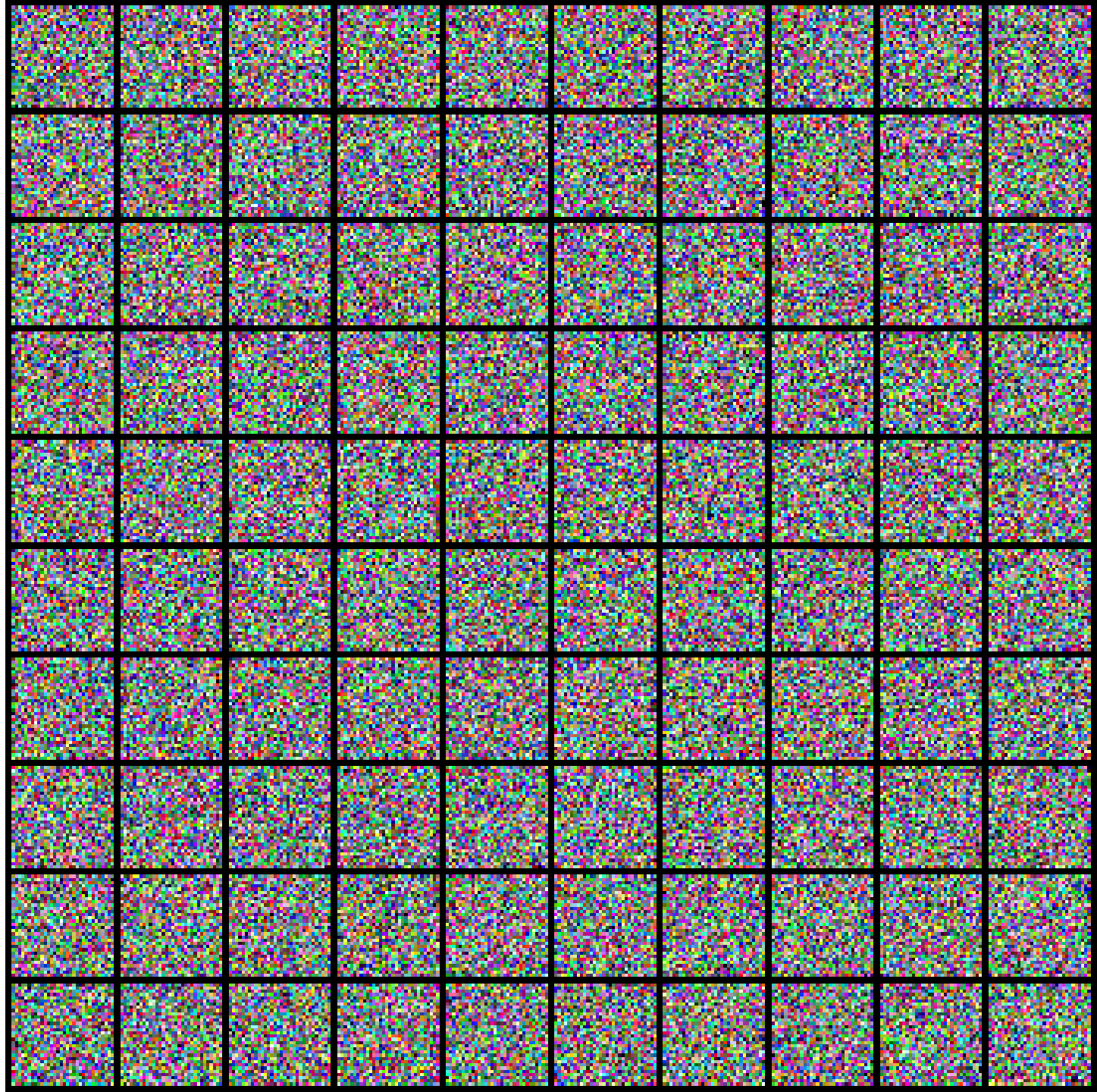
Amount of noise and number of steps determined by a *noise schedule* (hyperparameter)

Cosine Schedule (small amounts of noise first, then fast)



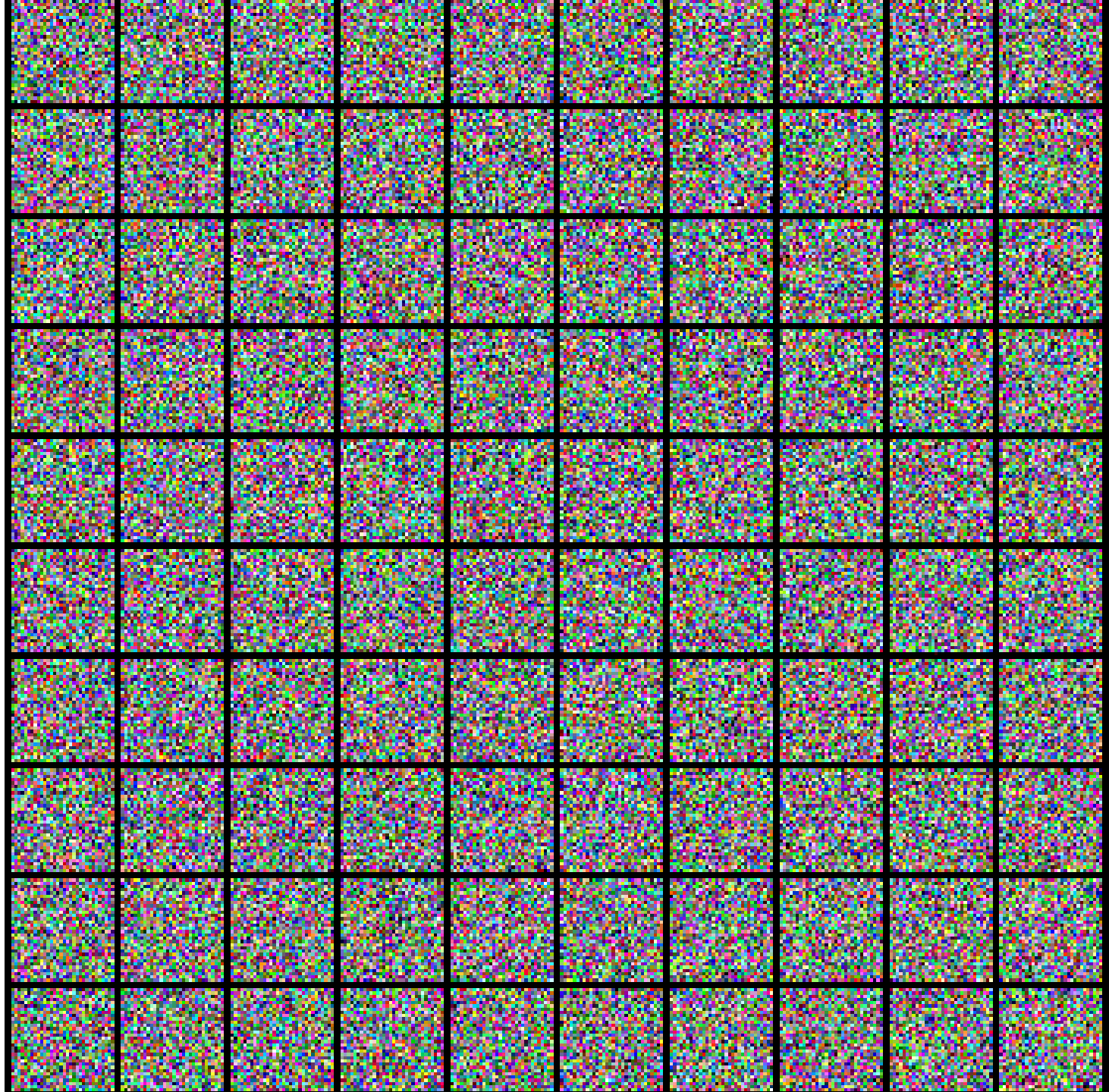
Examples

- Model trained on CelebA dataset



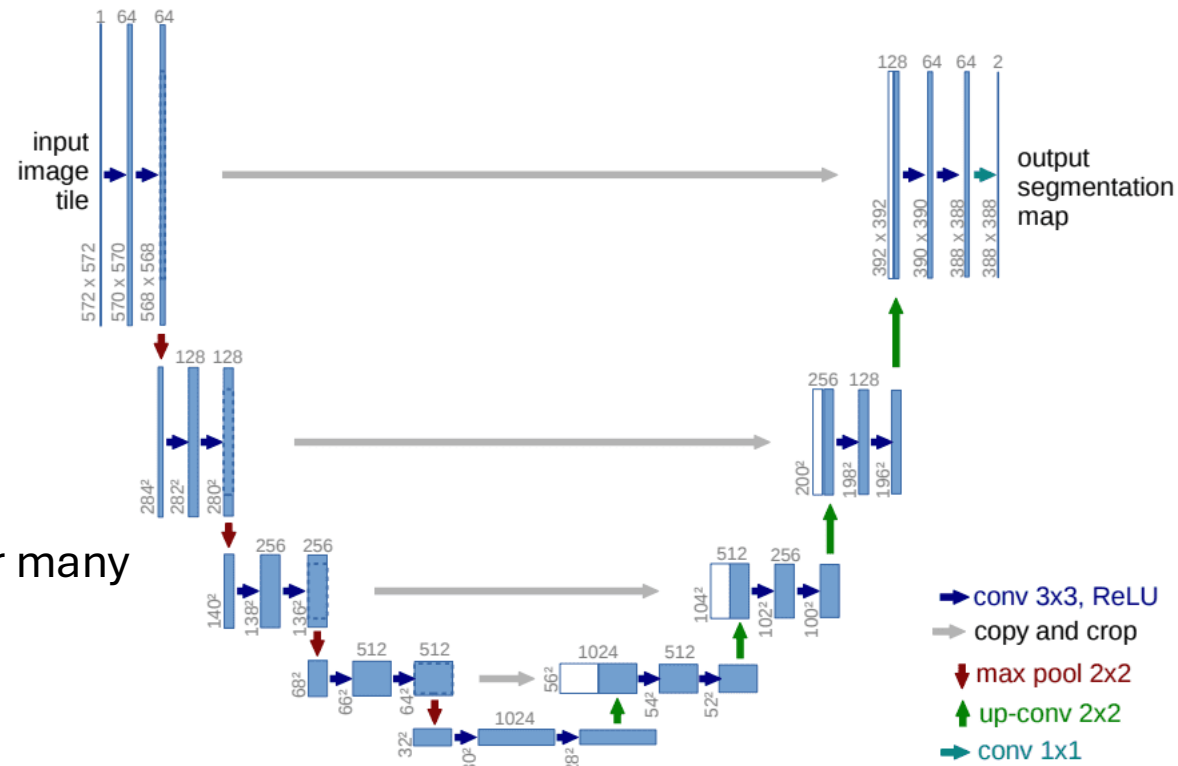
Examples

Model trained on CIFAR-10



Architectures

U-Net

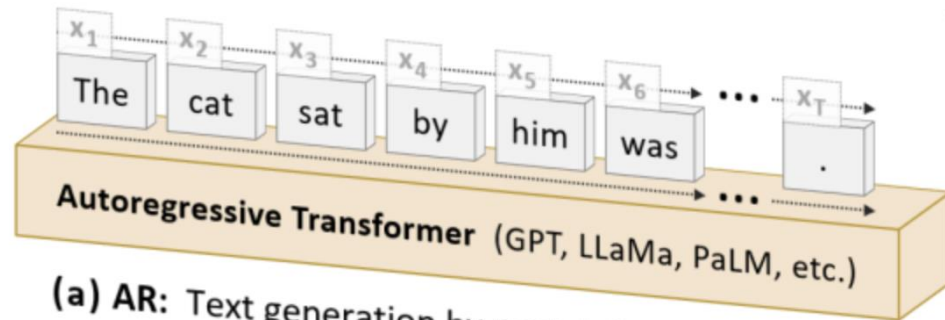


U-Net Architectures are used for many tasks:

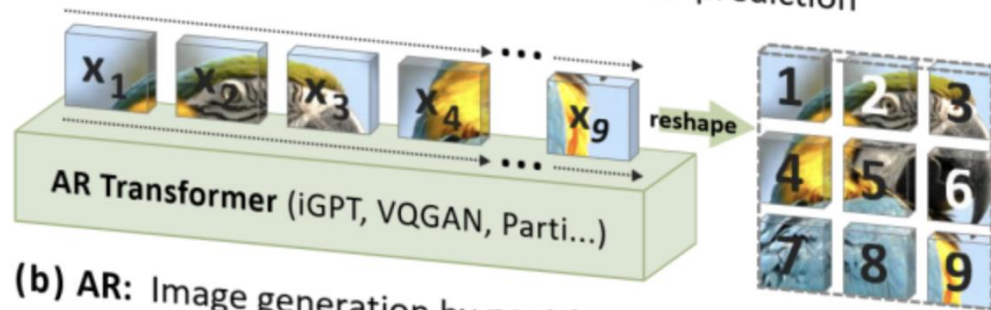
- Image segmentation
- Super-resolution
- VAE networks
- Etc.

Visual Auto-Regressive Generation

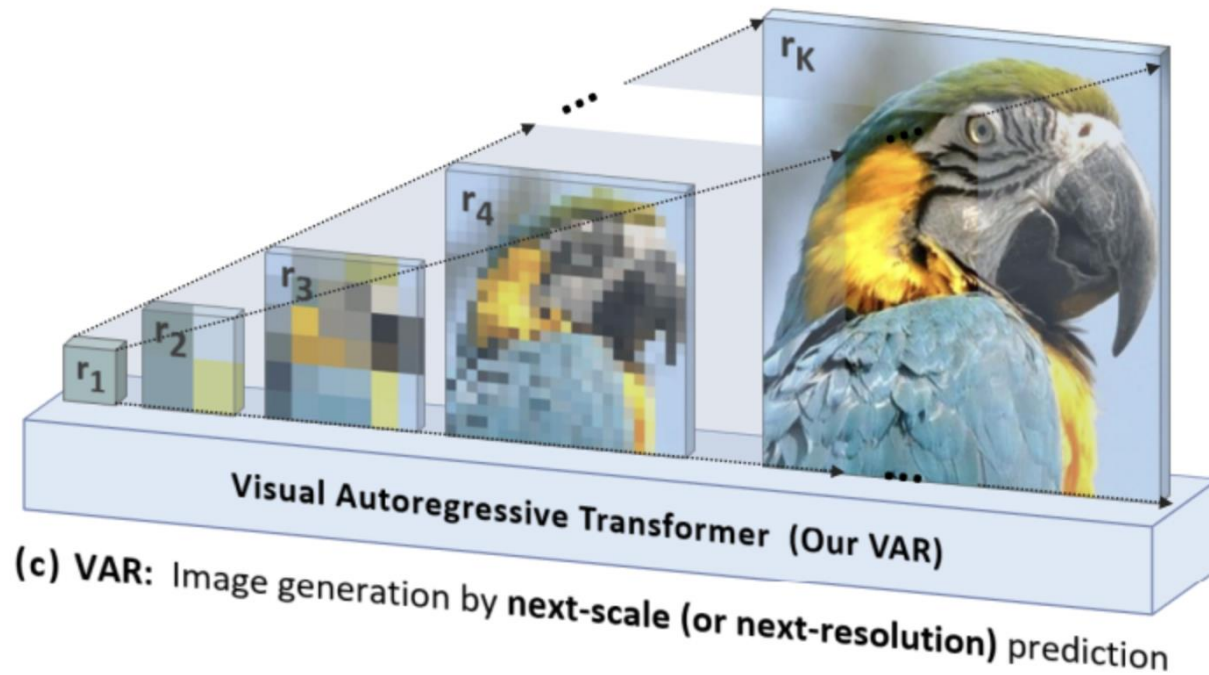
Three Different Autoregressive Generative Models



(a) AR: Text generation by **next-token** prediction



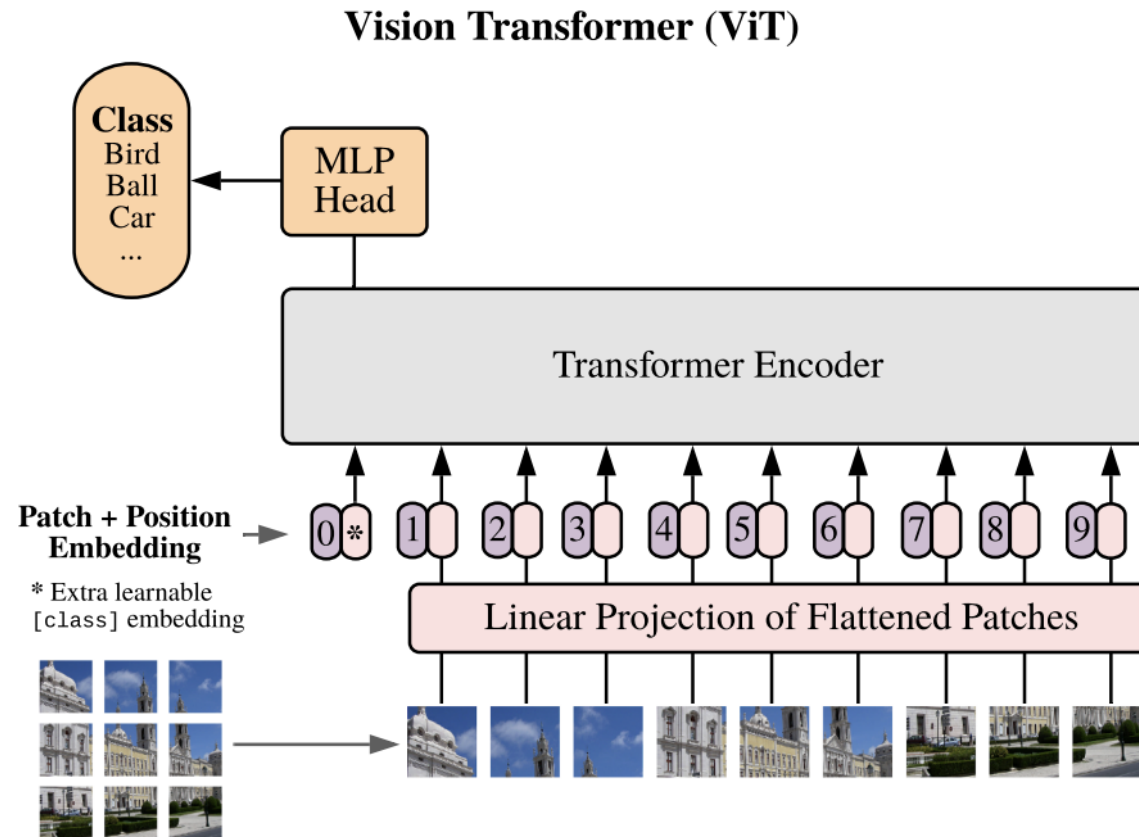
(b) AR: Image generation by **next-image-token** prediction



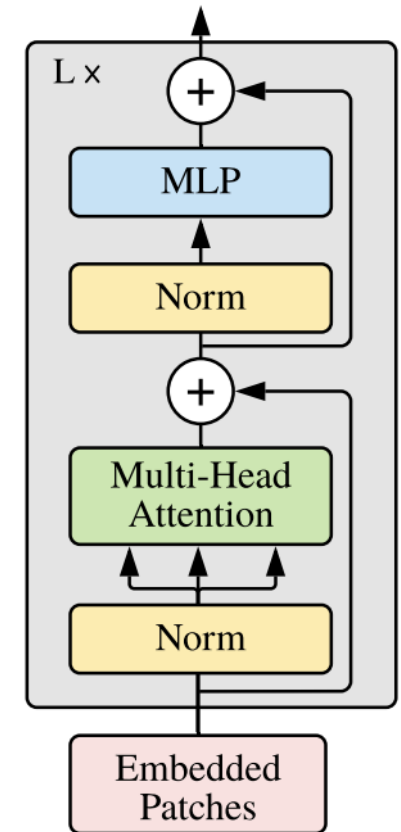
(c) VAR: Image generation by **next-scale (or next-resolution)** prediction

Vision Transformer (ViT)

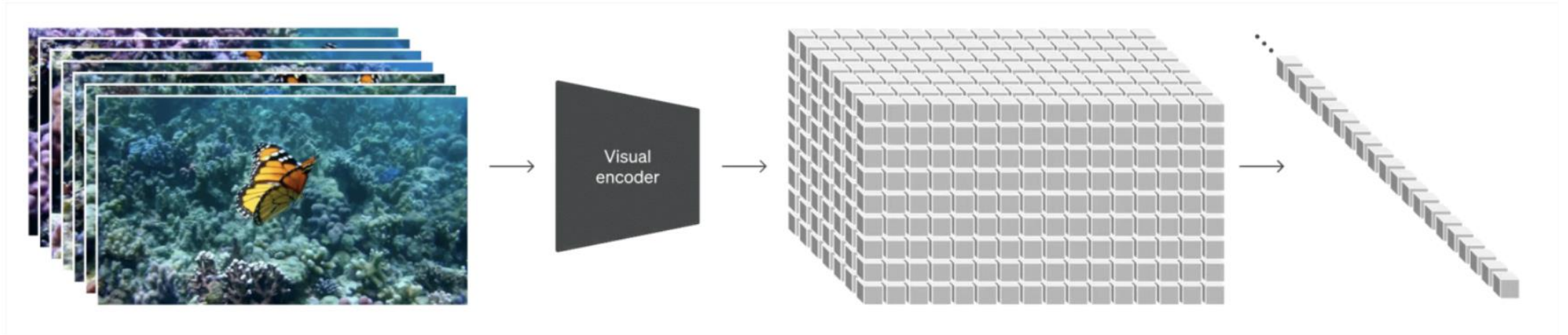
Image patches can be fed into transformers, like tokens in large language models



Transformer Encoder



Sora: SOTA Video Generation

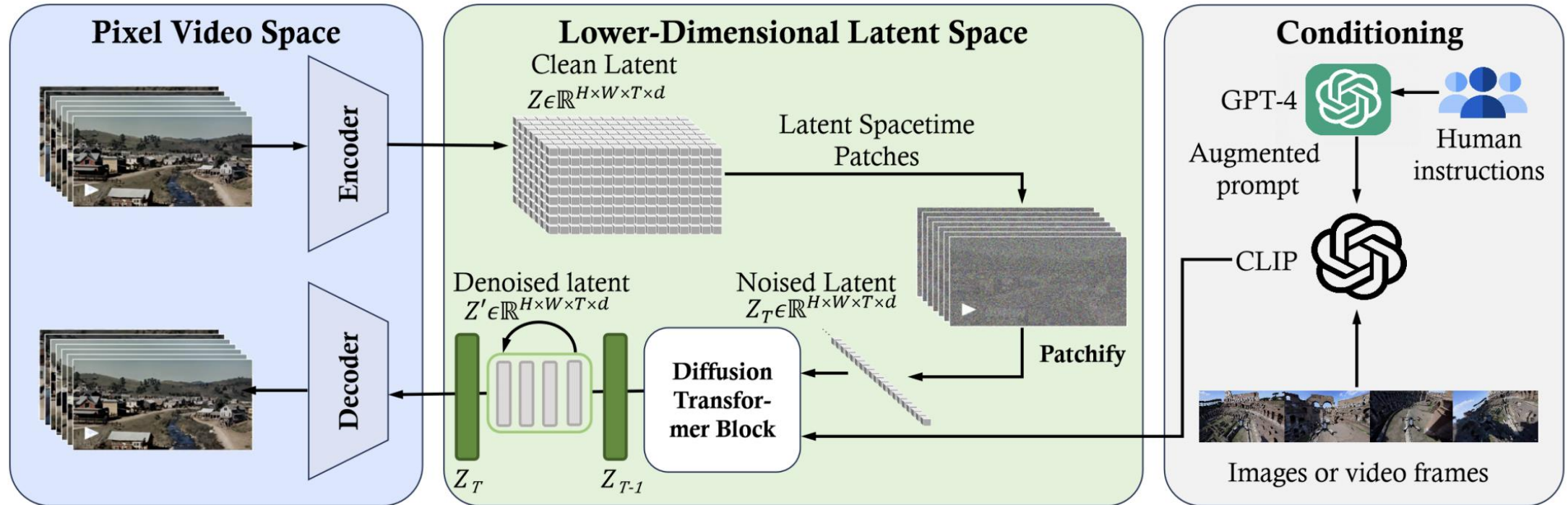


Visualization of Spacetime Patching (Processing) – Credit: OpenAI (Sora)

Sora uses *spacetime patches* as the basis for their input

Spacetime patch: a small patch over multiple frames (i.e., has a height, width, and temporal dimension)

Video Diffusion Models



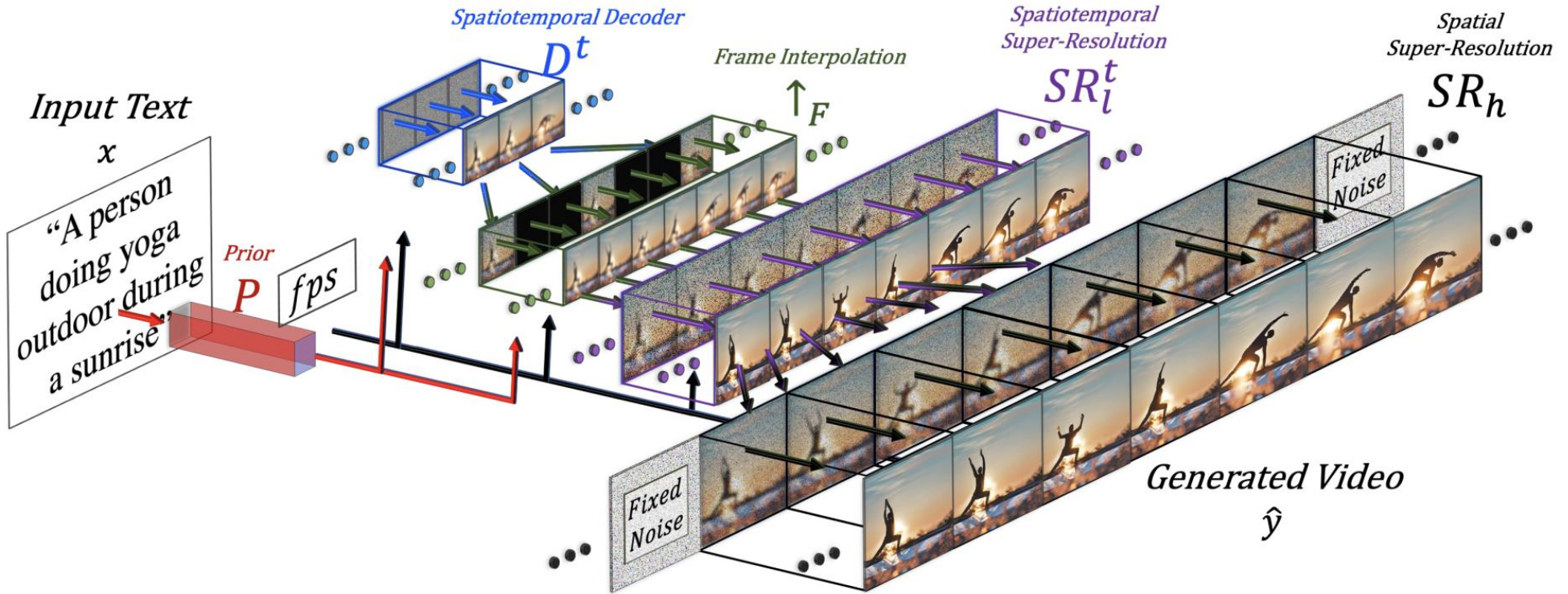


Figure 2: **Make-A-Video high-level architecture.** Given input text x translated by the prior P into an image embedding, and a desired frame rate fps , the decoder D^t generates 16 64×64 frames, which are then interpolated to a higher frame rate by $\uparrow_F$, and increased in resolution to 256×256 by SR_l^t and 768×768 by SR_h , resulting in a high-spatiotemporal-resolution generated video $\hat{y}$.

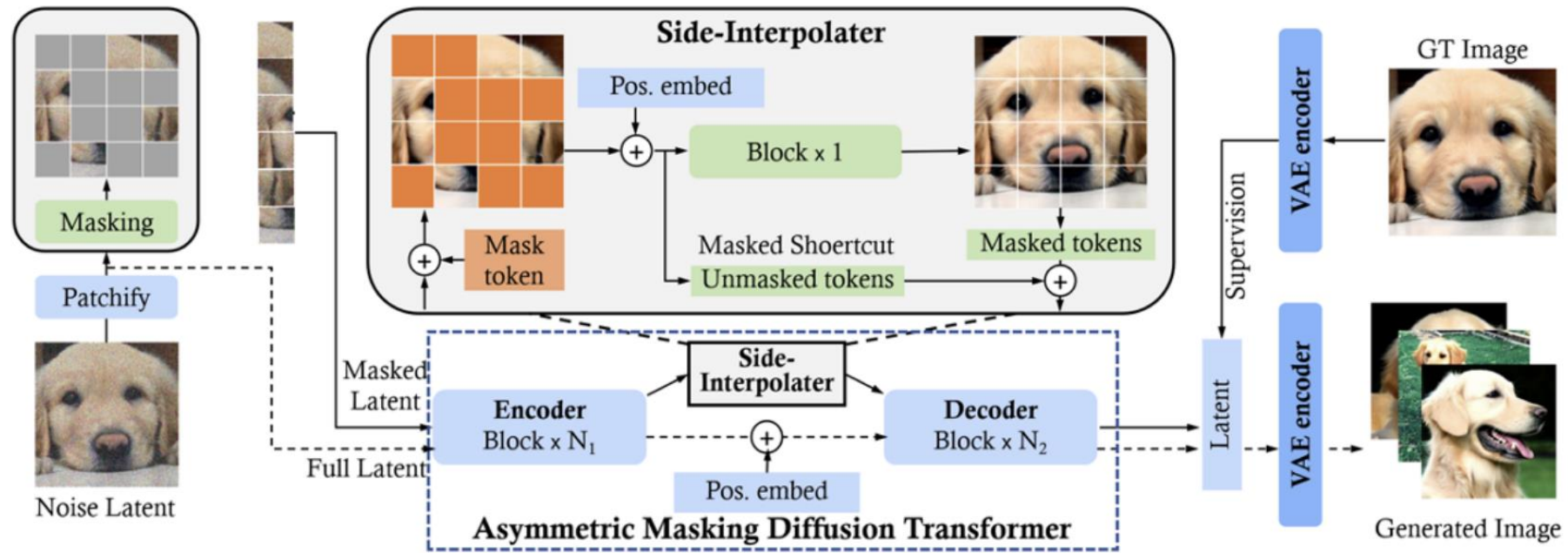


Figure 12: The overall framework of Masked Diffusion Transformer (MDT). A solid/dotted line indicates the training/inference process for each time step. Masking and side-interpolator are only used during training and are removed during inference.

Comparison of Image Generation Methods

VAEs

Advantages: Fast sampling process, stable training, interpretable latent space
Disadvantages: lowest quality option in general, hard to balance objective

GANs

Advantages: Fast sampling process, better outputs than VAEs,
Disadvantages: Can suffer from mode collapse, unstable training

Diffusion Models

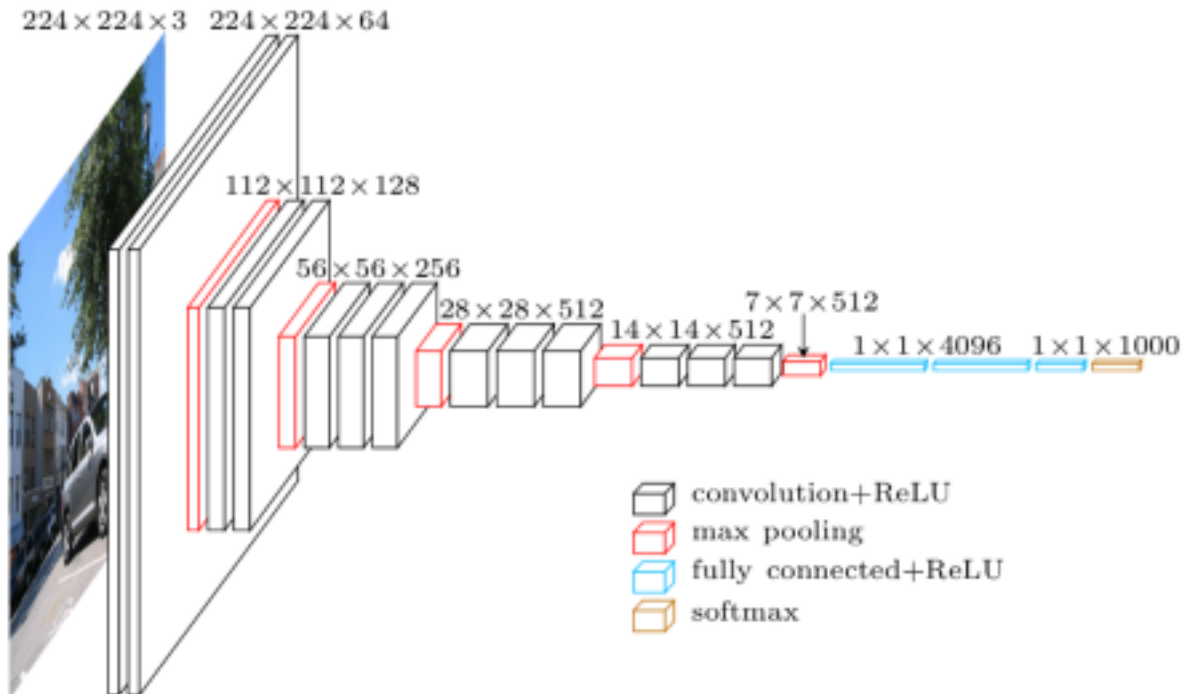
Advantages: Stable training, high quality outputs, and good “mode coverage”
Disadvantages: Requires lots of resources, slow sampling process

Geometric Deep Learning

Structured Data

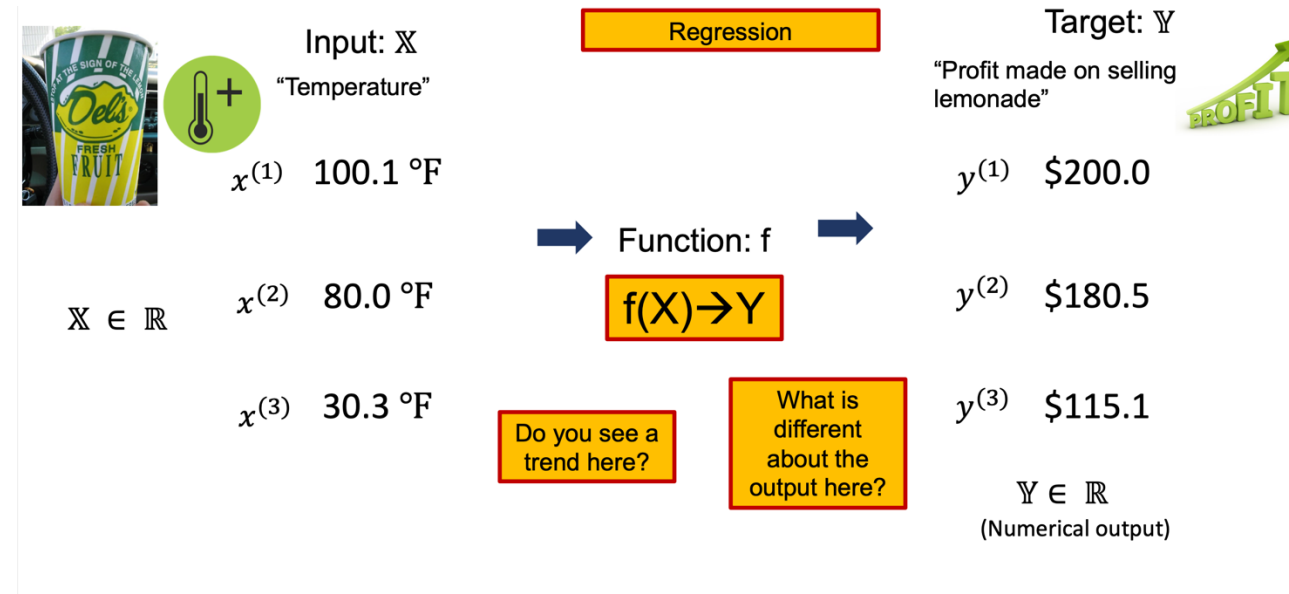
Images are a **structured** type of data

The structure (order and layout) of pixels matters

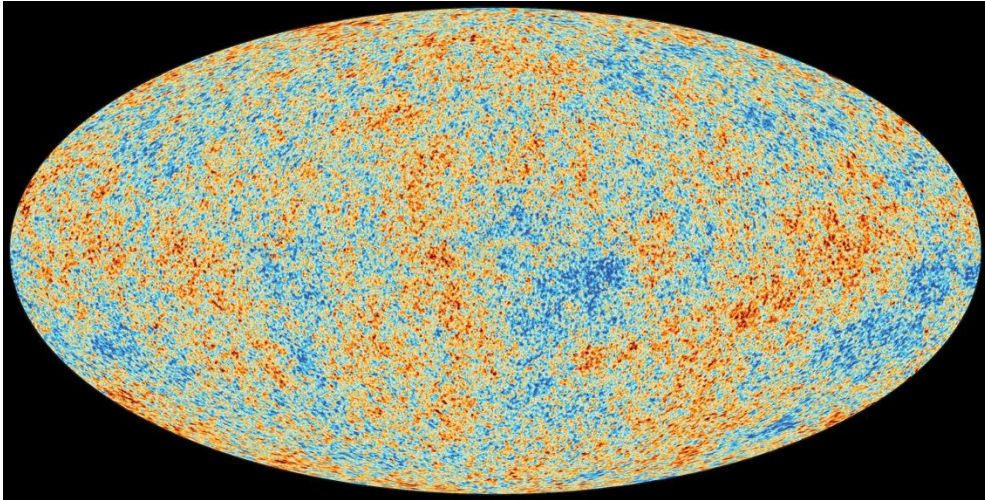


Lemonade stand data is **not structured**

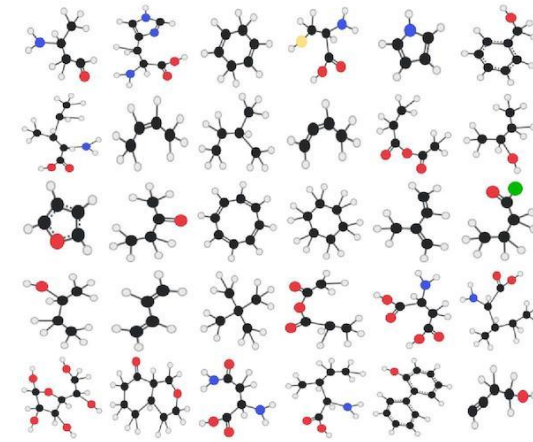
The order of features we use does not change training



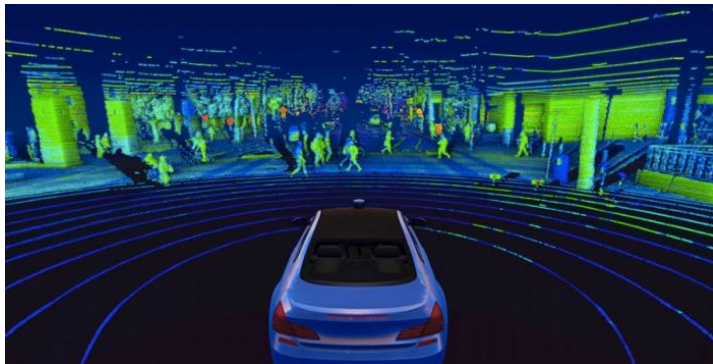
Other Types of Structured Data Types



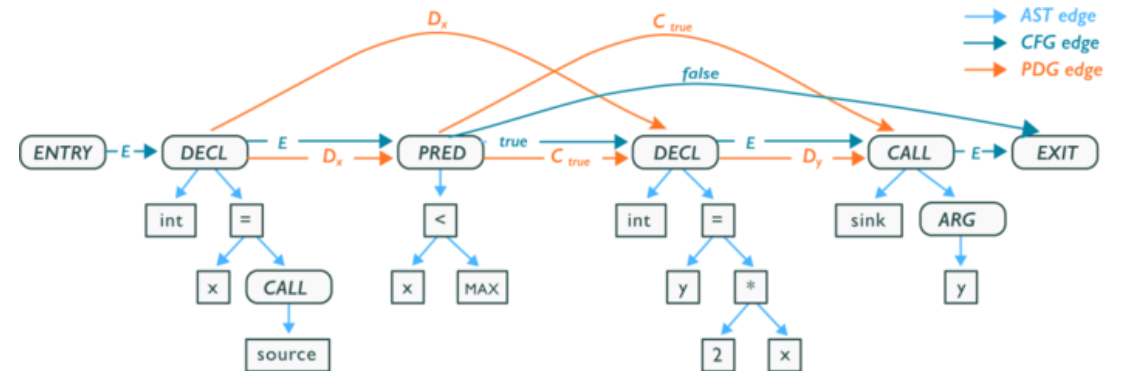
Microwave Background Radiation



Molecule Data



Point Clouds (from LIDAR)



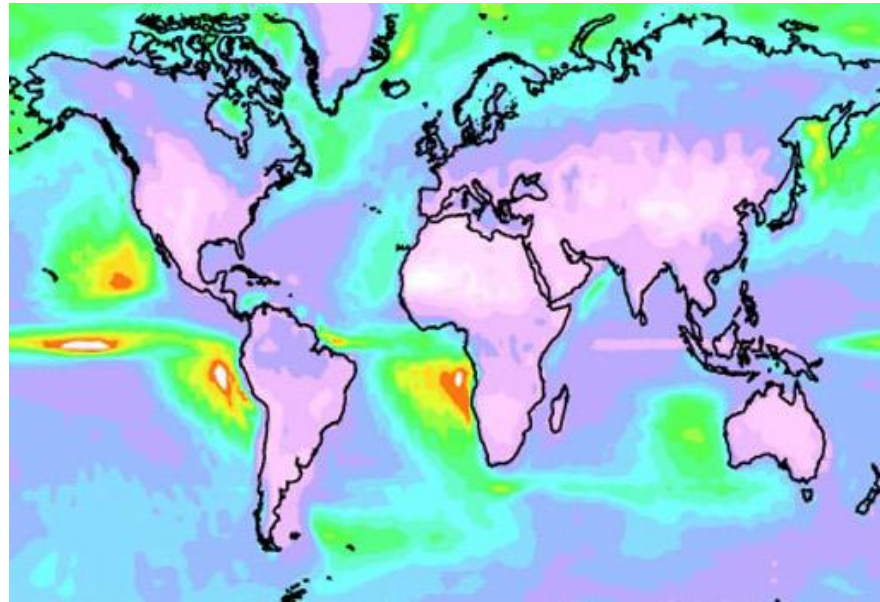
Code Graphs

Geometric Deep Learning

- We turned to convolutions for image data to give our networks “spatial reasoning”
- By Explicitly modelling the relationship of our data, we can achieve better results
- Geometric Deep Learning is the subfield of DL dedicated to learning representations of *structured* data.

Goal: Take as input data from satellites, predict carbon dioxide

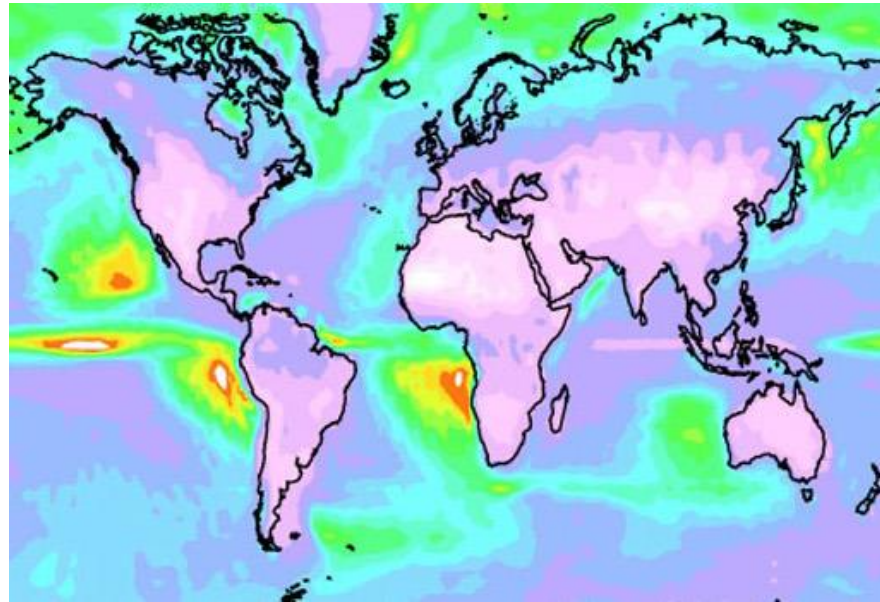
Idea 1: Take all of the data as input, make it look like an image, use CNNs to learn to output carbon dioxide amounts



Goal: Take as input data from satellites, predict carbon dioxide

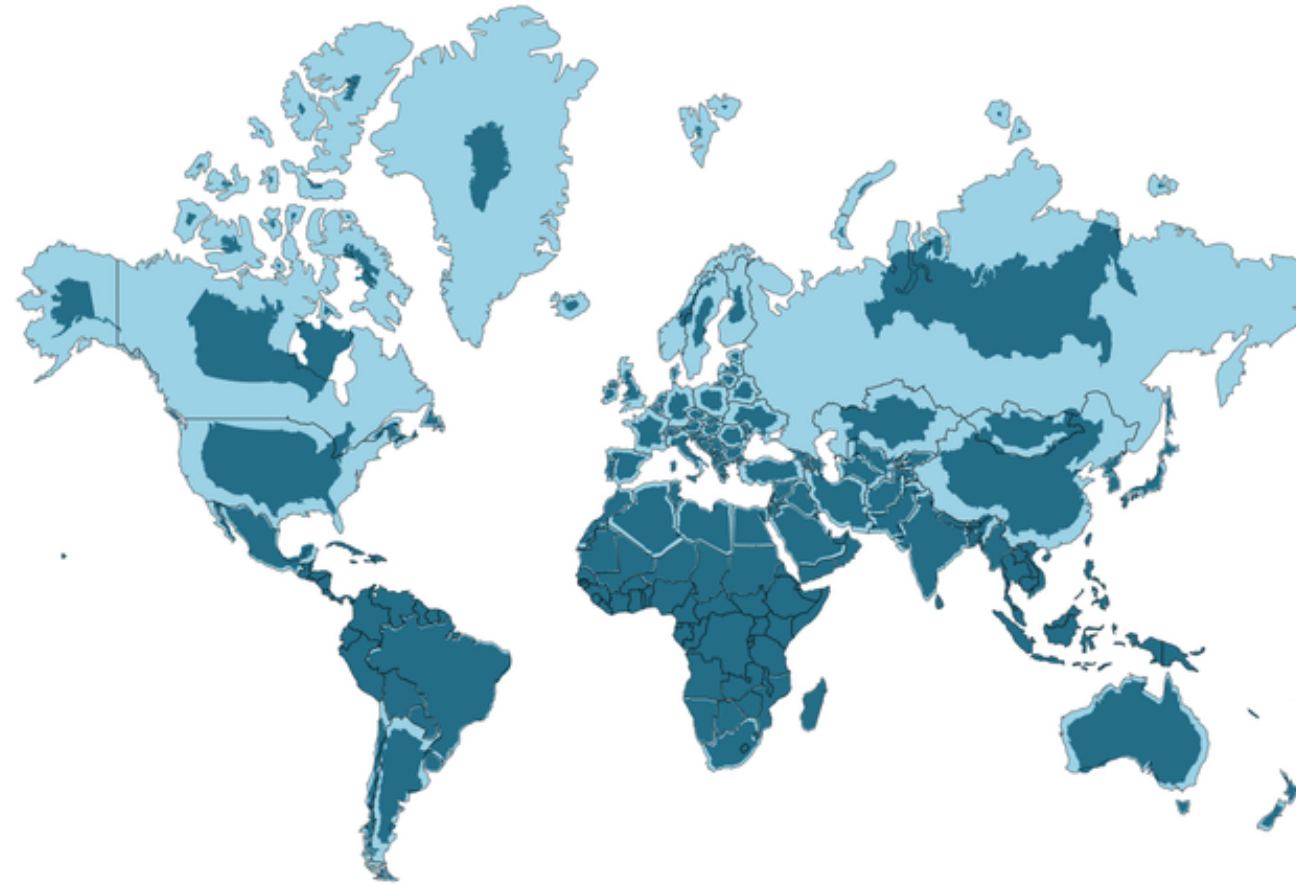
Idea 1: Take all of the data as input, make it look like an image, use CNNs to learn to output carbon dioxide amounts

Issue: Maps 😞



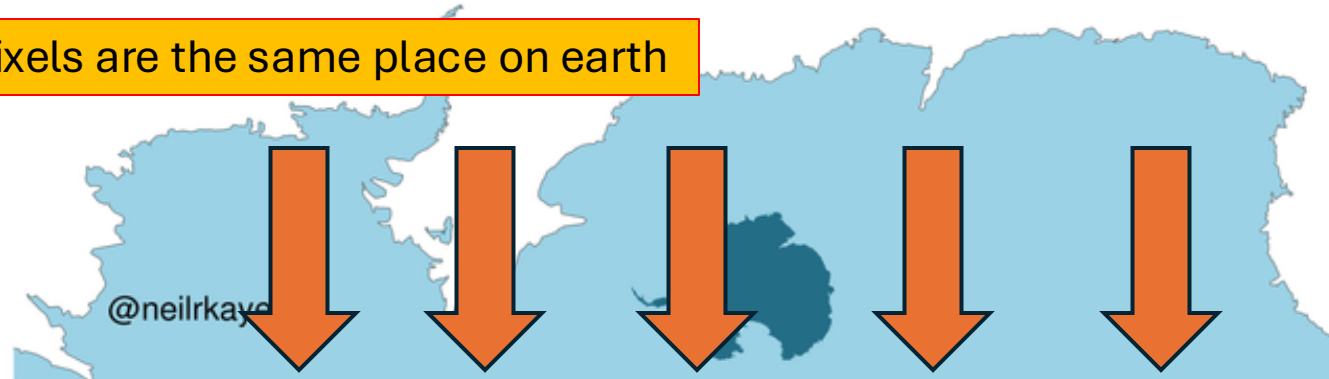
Issue: Trying to represent a 3D sphere on a 2D plane

A 2D convolution doesn't make sense here...
But there's still structure (don't just want an MLP)
What about a Spherical Convolution!



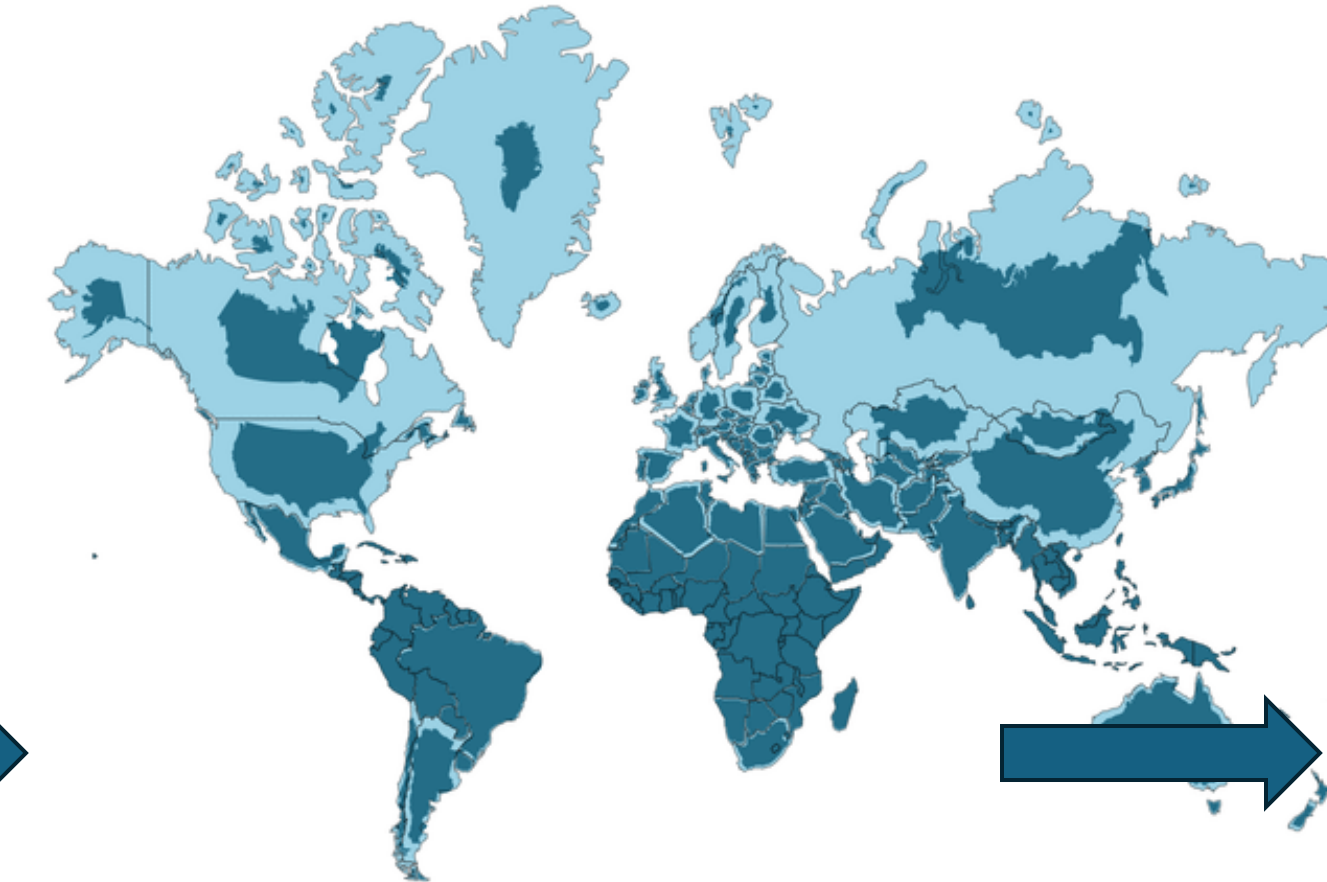
All these pixels are the same place on earth

What should we pad the side of the image with?



Instead of padding with 0's, "pad"
with adjacent locations on earth

Convert 2D convolution on
grids to work on Spheres

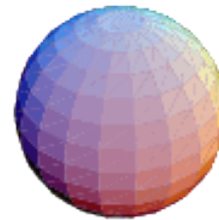


Manifolds

- Topological Space that looks locally like Euclidean space, but can be globally curved
 - For example, the Earth
- Can we generalize convolutions to more than just spheres?

Need to specify which points are “close” to other points, convolve neighborhoods of points together

sphere



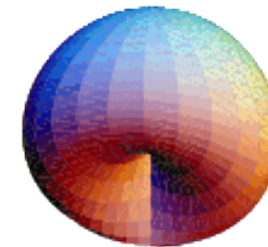
torus



double torus



cross surface



Klein bottle

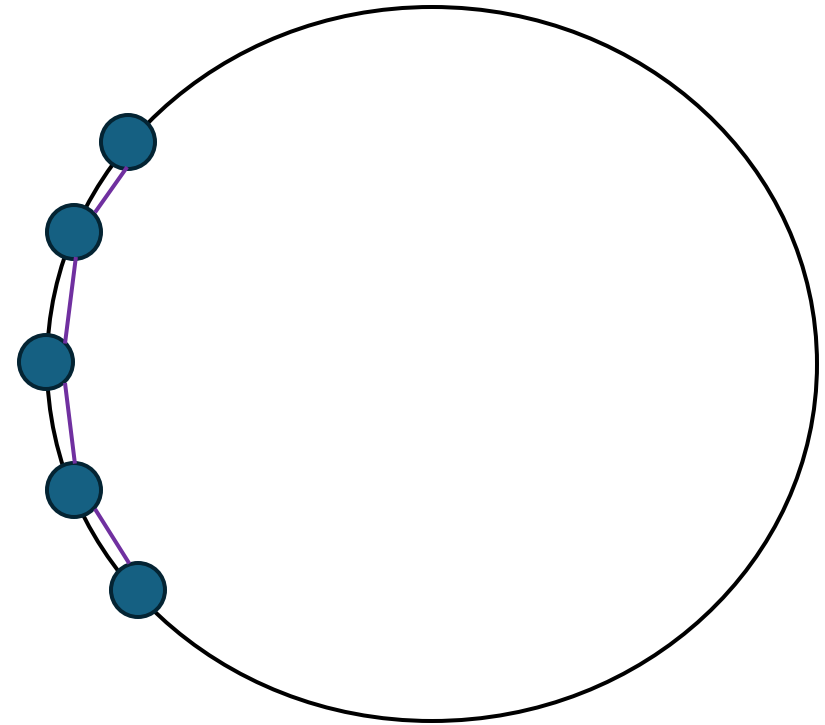


Extending Convolutions to Manifolds

A circle is a manifold in 2D
(like a sphere or torus in 3D)

For every point on the manifold,
connect it to “close” points

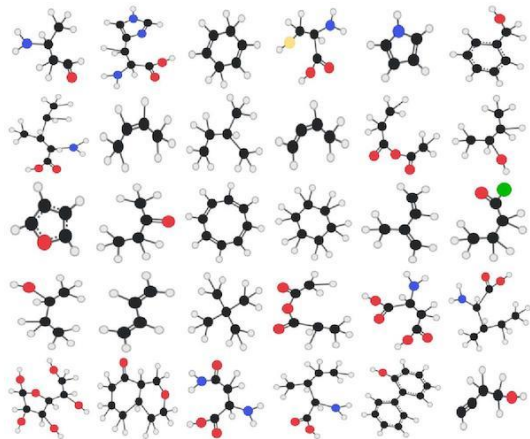
What data structure does this lead to?



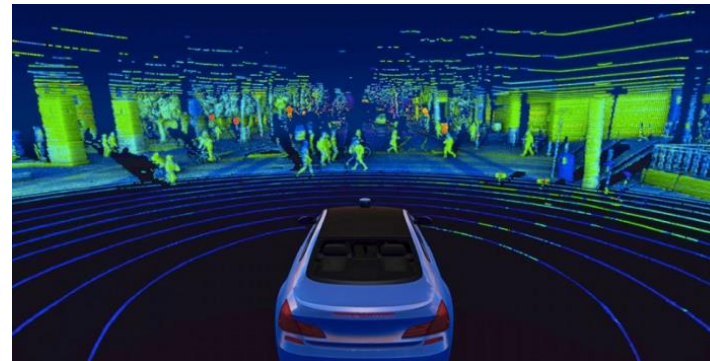
Geometric Deep Learning

- It is often confusing why much of Geometric Deep Learning deals with graphs...
 - Geometry happens in continuous space
 - Graphs are discrete

Molecules



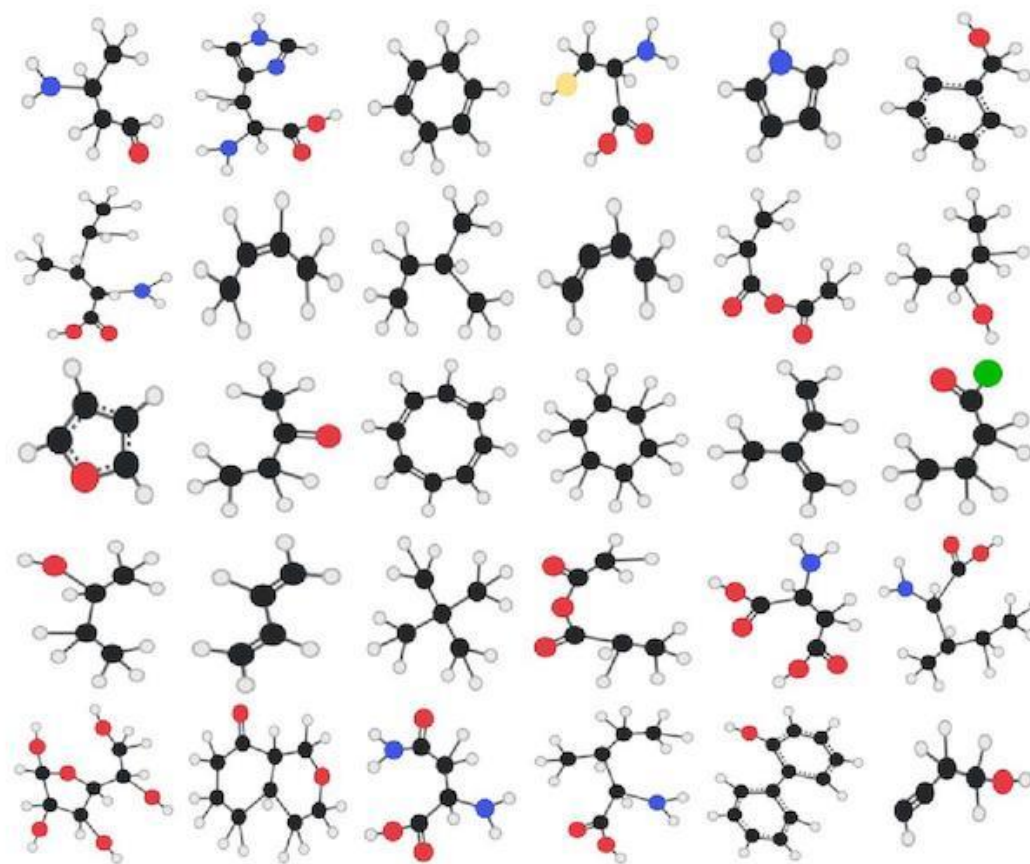
LIDAR Data



What's Hard About Neural Networks for Graphs?

- Can have different numbers of nodes
- Can be arbitrarily complex
 - Different numbers of edges
 - Different edge patterns
 - Weighted edges

How will a Graph Convolution have to differ from a 2D convolution?

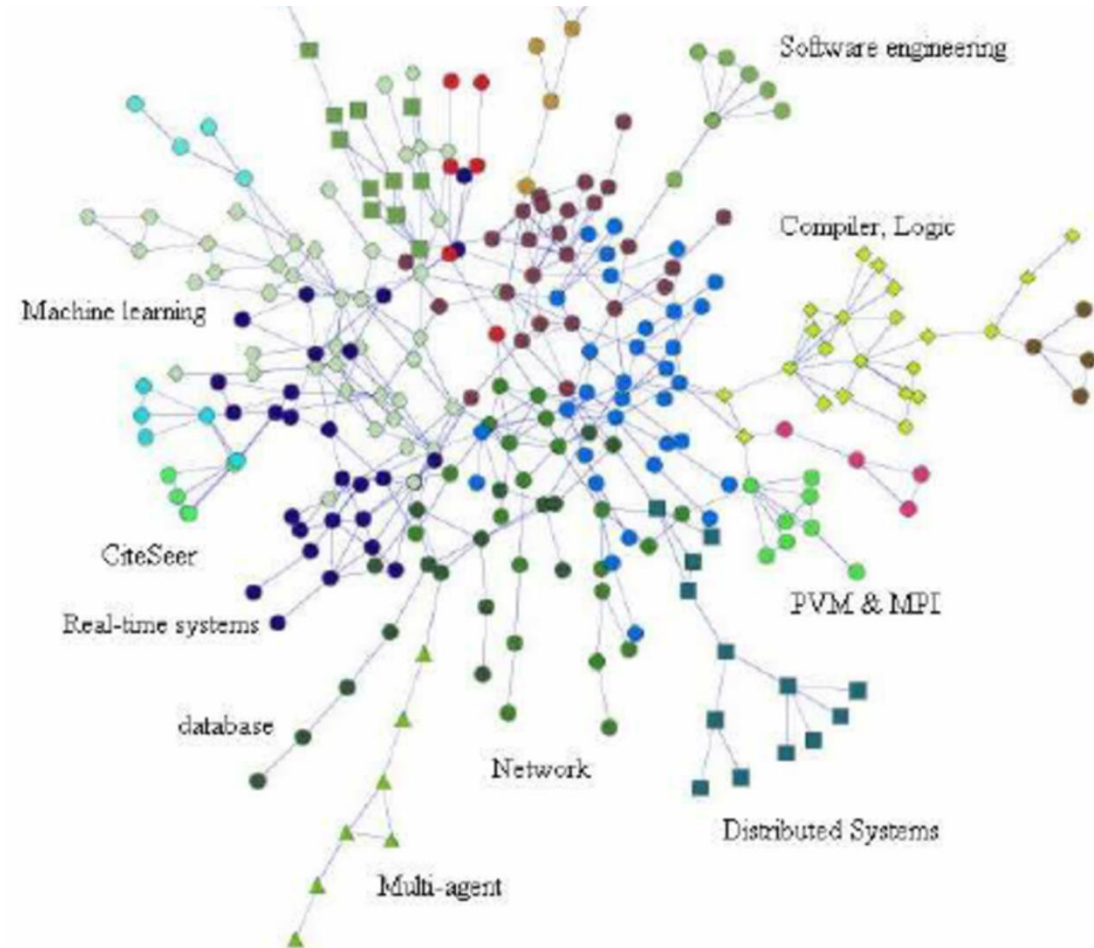


Example Dataset

CiteSeer Dataset:

Collection of academic papers:

- directed edges connect papers to the papers they cite
- Words used by each paper
 - Vector of length 3703 for each word in “vocabulary”, 1 if present 0 if not



Graph Convolutions: Message Passing

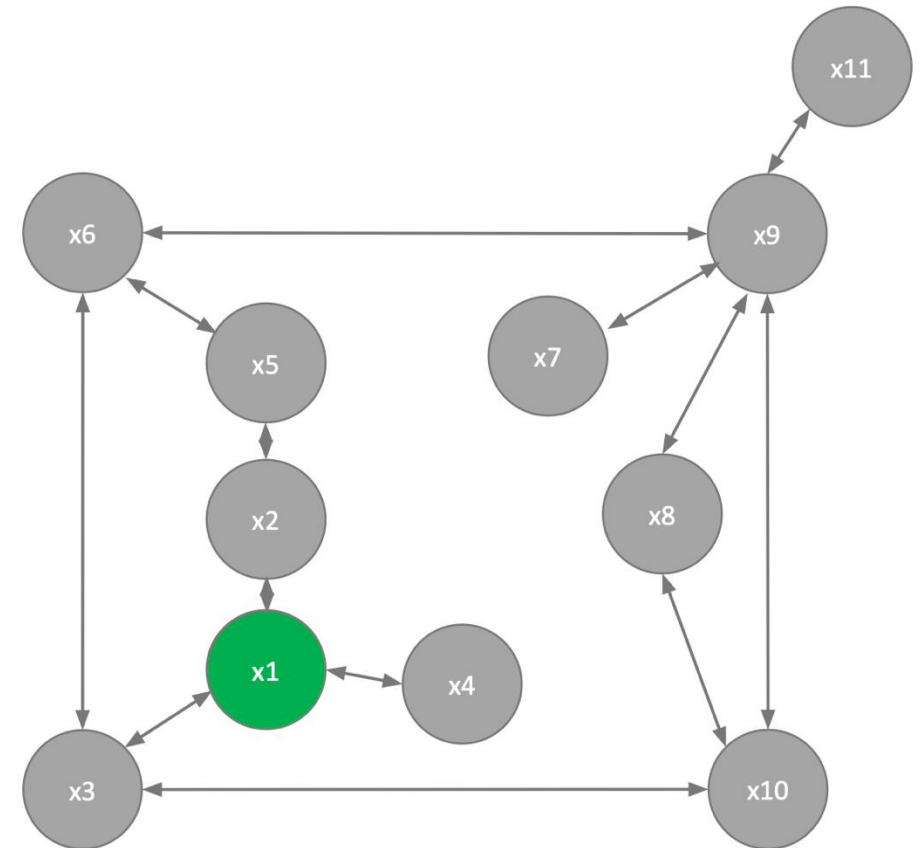
- Graph Neural Networks (GNNs) need to share information between connected nodes
- This sharing of information is called *message passing*

Graph Convolutions

For a graph $G=(V, E)$

Each node v_i has features x_i , each edge (v_i, v_j) can also have features (e.g., weight)

1. Start by mapping x_i to hidden features h_i
 $f_{init}(x_i) = h_i$

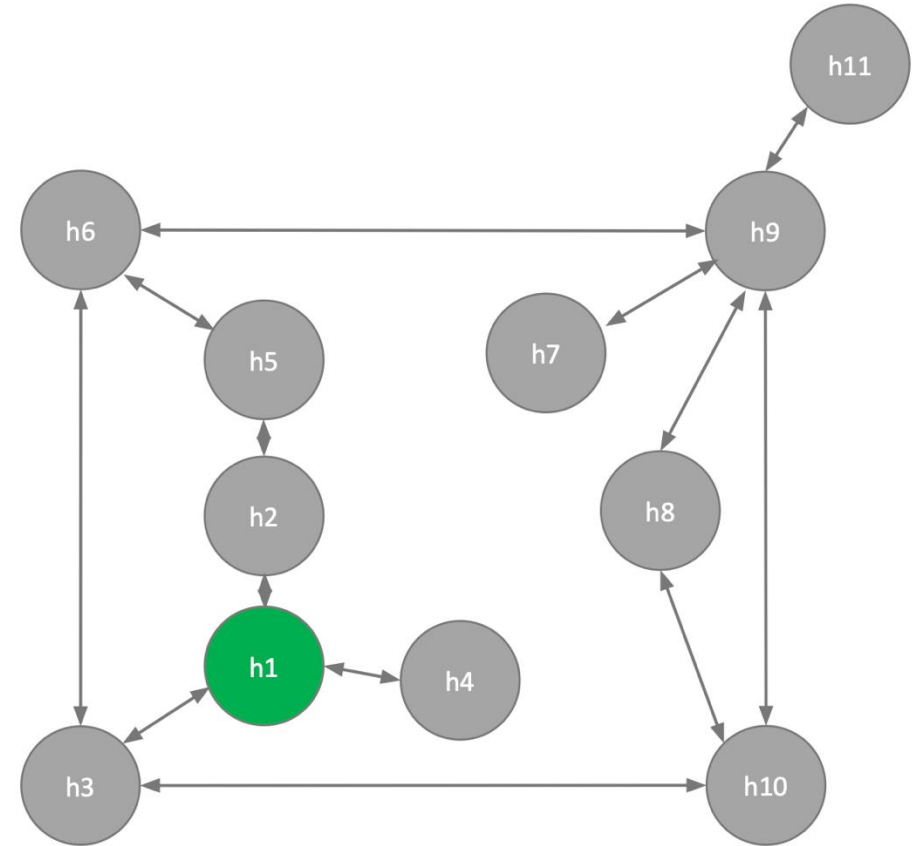


Graph Convolutions

For a graph $G=(V, E)$

Each node v_i has features x_i , each edge (v_i, v_j) can also have features (e.g., weight)

1. Start by mapping x_i to hidden features h_i
 $f_{init}(x_i) = h_i$

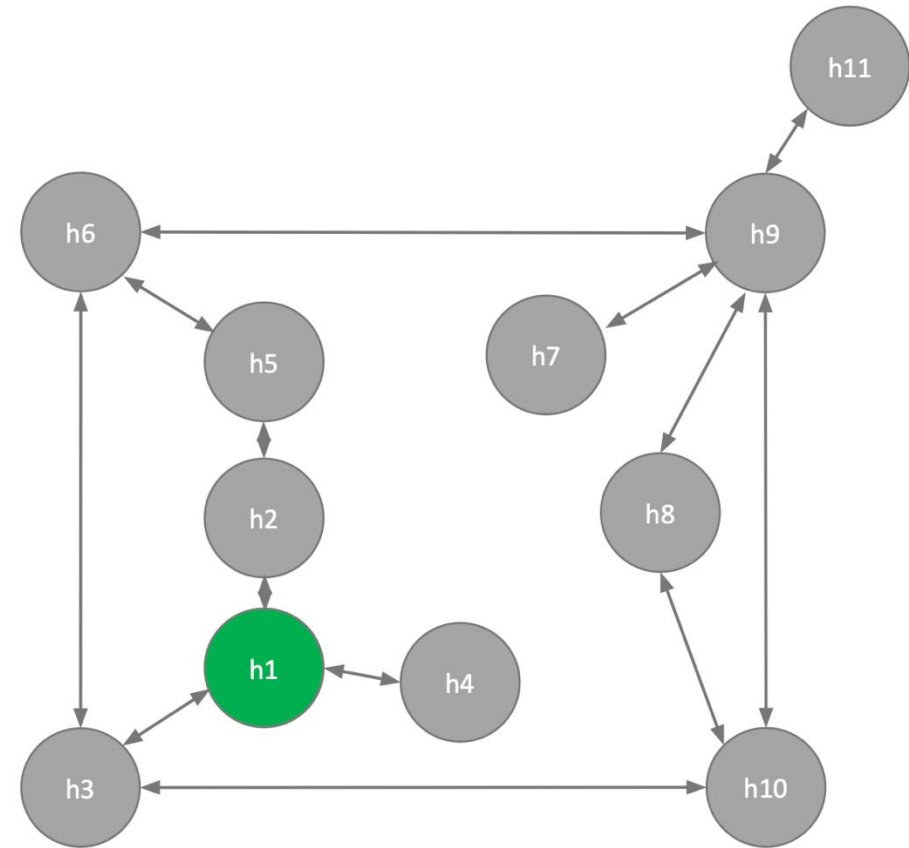


Message Passing

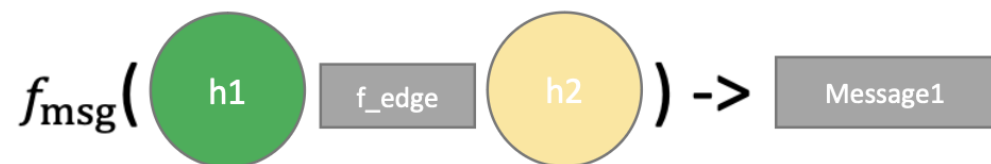
For a graph $G=(V, E)$

Each node v_i has features x_i , each edge (v_i, v_j) can also have features (e.g., weight)

1. Start by mapping x_i to hidden features h_i
 $f_{init}(x_i) = h_i$
2. Perform message passing among adjacent nodes

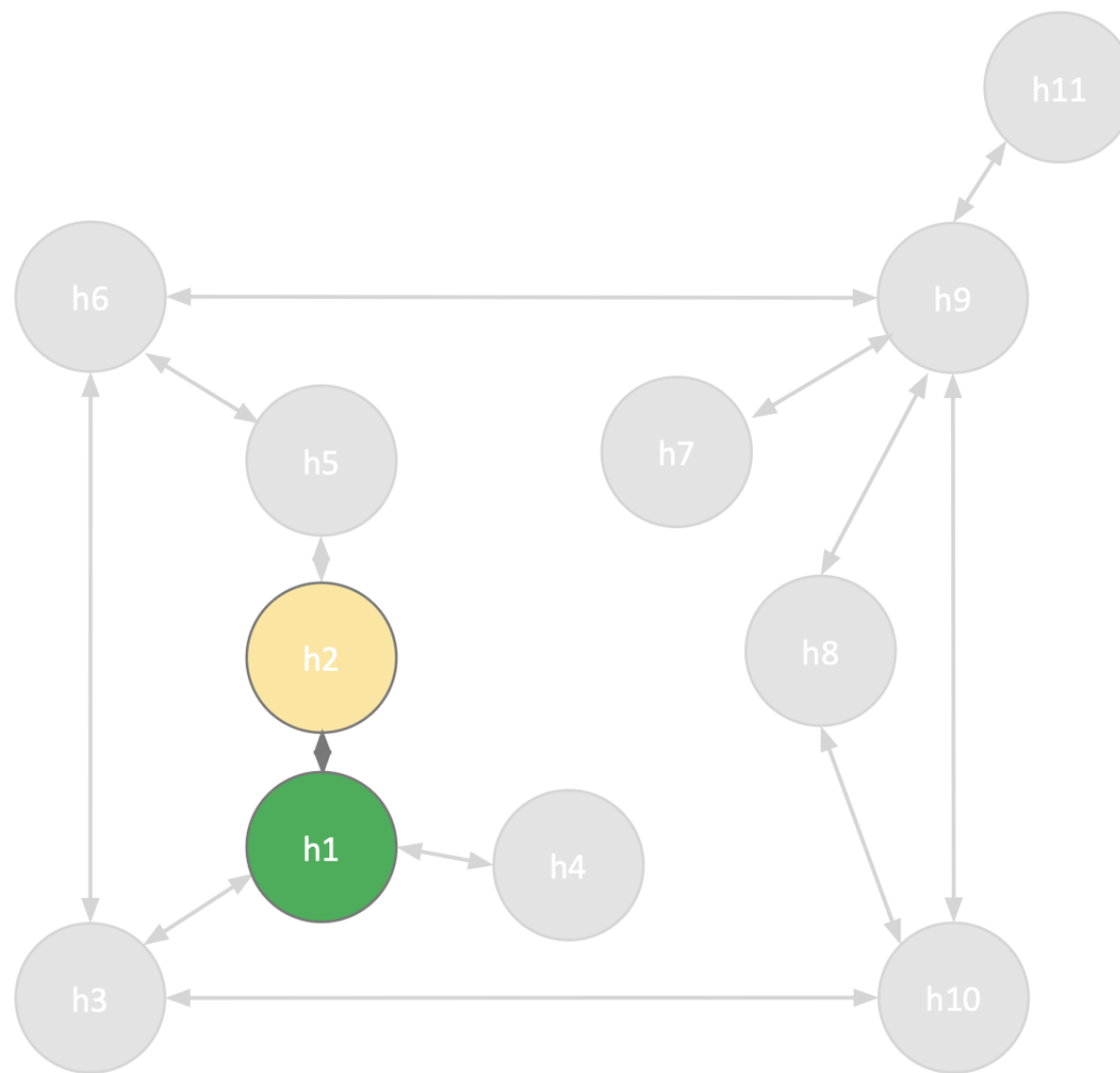


Message Passing

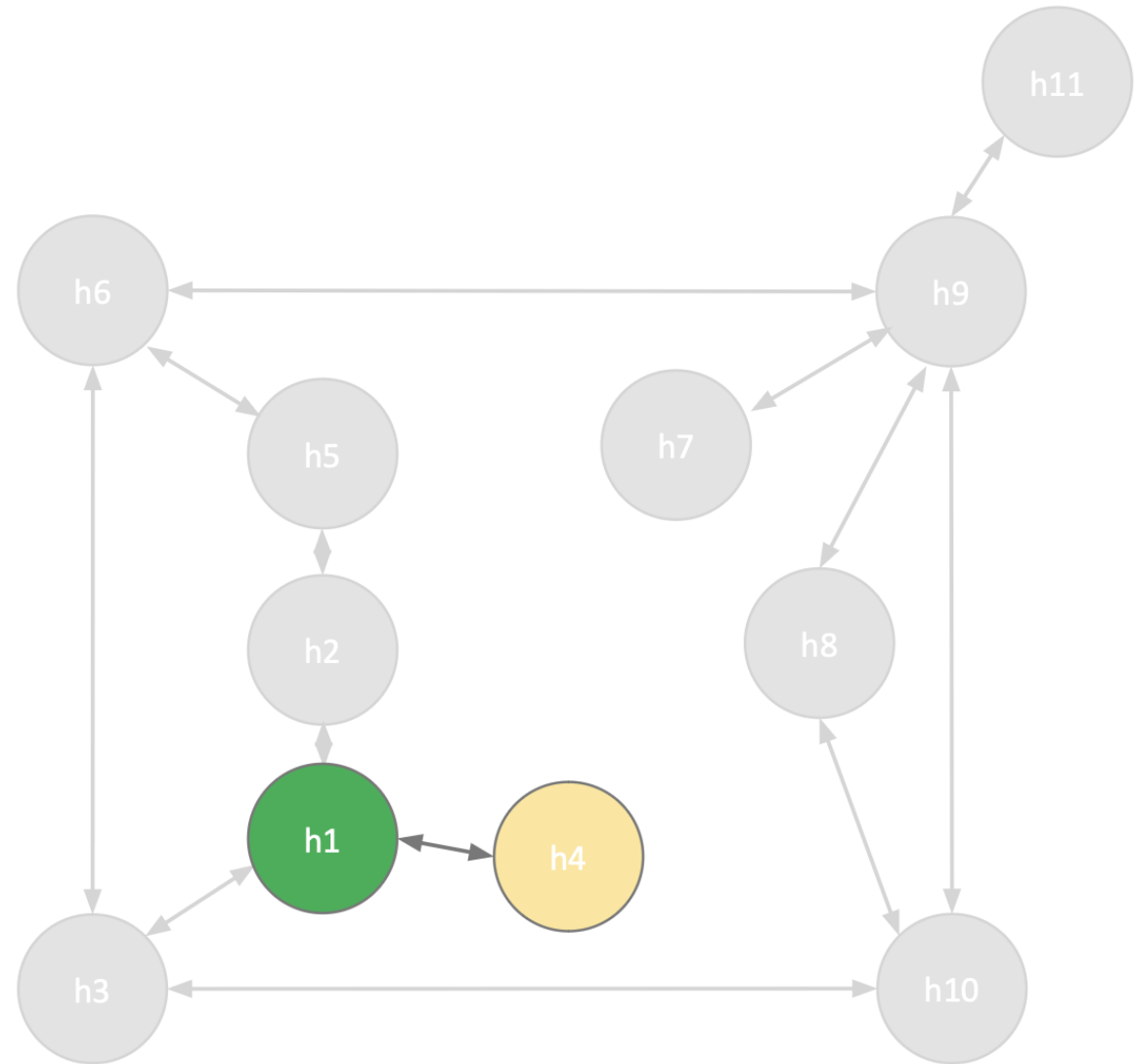
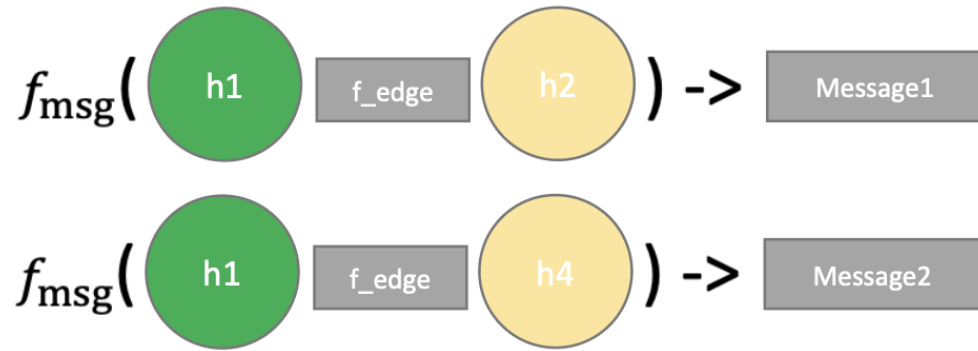


We use a neural network f_{msg} to compute a **message** between two nodes in the graph

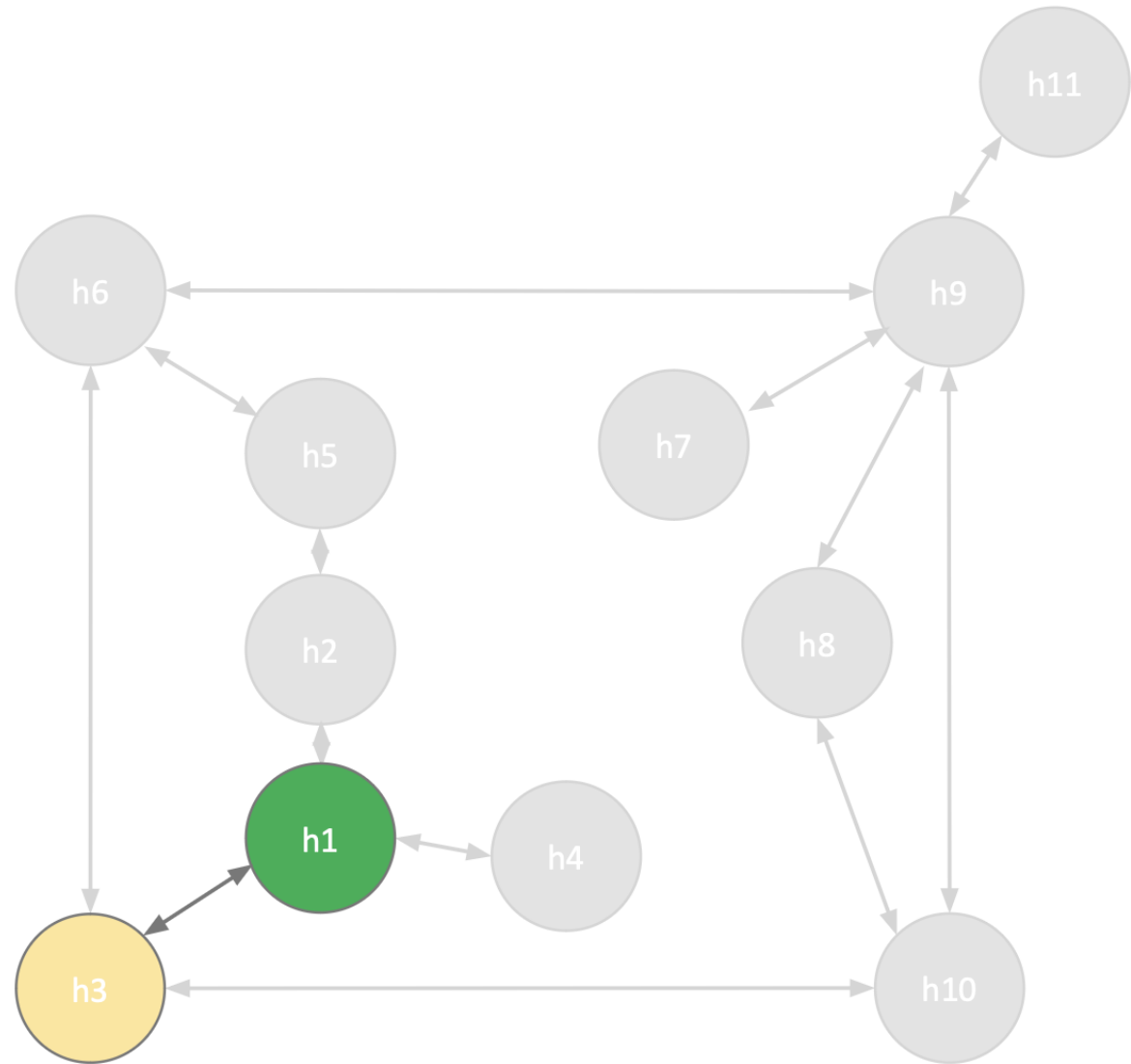
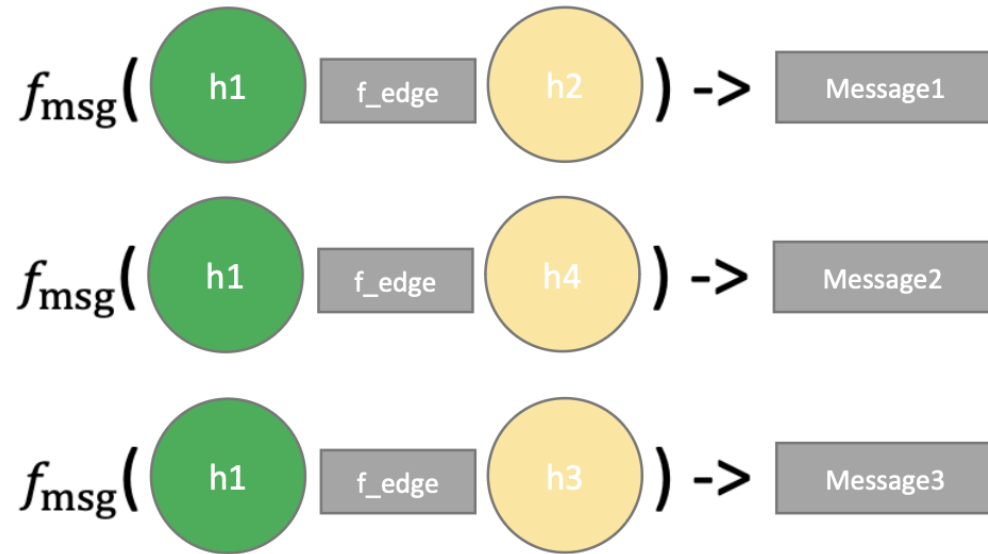
- Inputs
 - Hidden states for each node
 - Features along the edge
- Output
 - Message vector



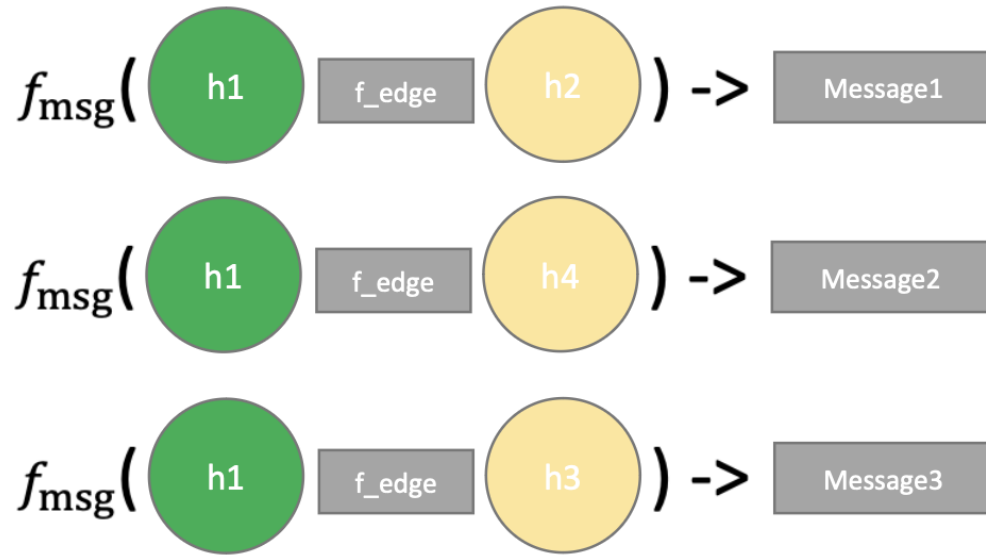
Message Passing



Message Passing

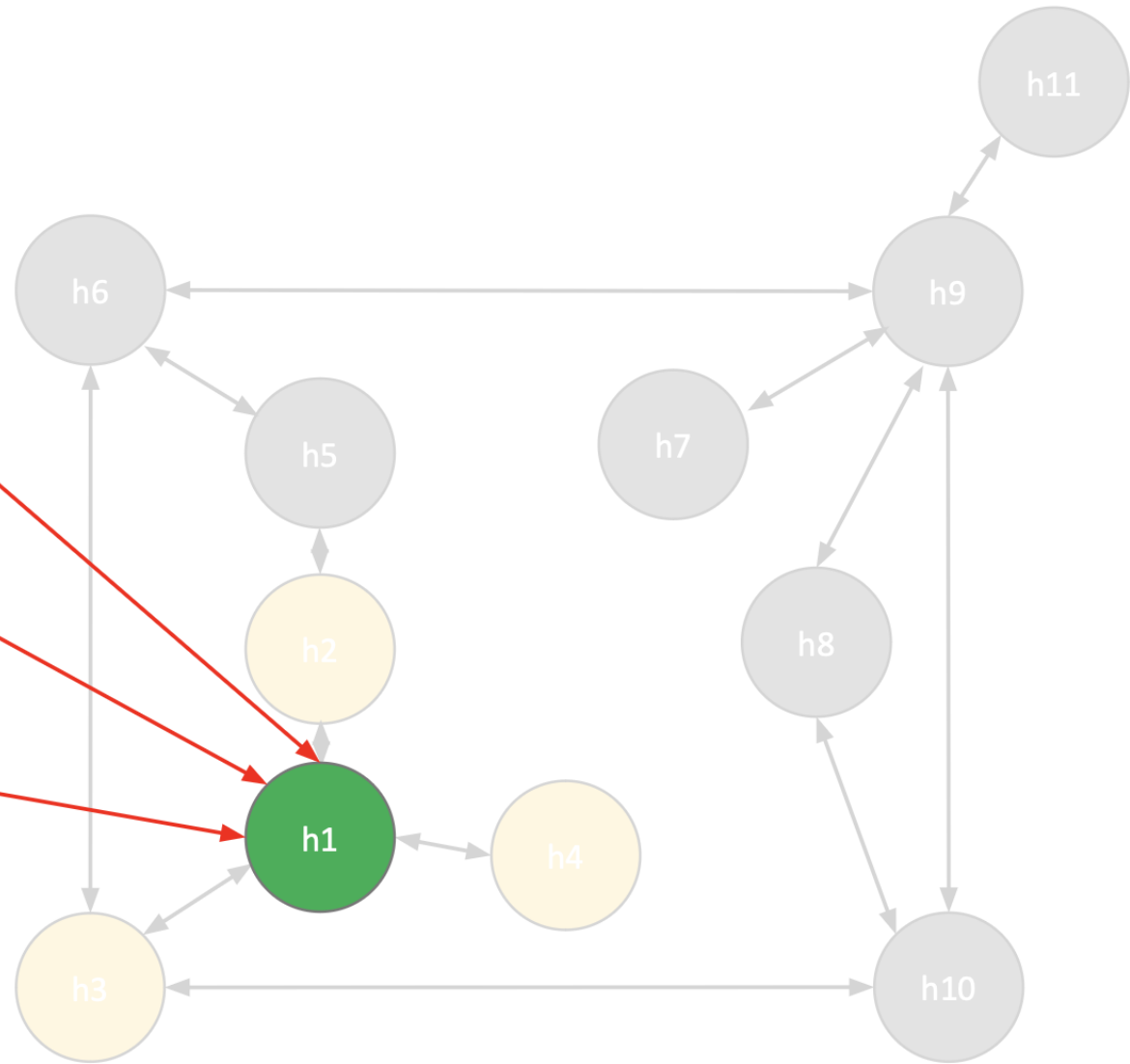


Message Passing



Aggregate all the messages to compute the output hidden state vector for h_1

(e.g. sum up the messages)



NOTE: The aggregation function can differ leading to different formulations of GNNs

GraphSage

Average of neighbor's embeddings for previous round

Learnable weights B times previous embedding

$$h_v^k = \sigma \left(W_k \sum_{u \in N(v)} \frac{h_u^{k-1}}{|N(v)|} - B_k h_v^{k-1} \right) \forall k \in \{1, \dots, K\}$$

Hidden features after k rounds of message passing

Weights layer for k'th round

Nonlinear activation function (e.g., ReLU)

$$z_v = h_v^K$$

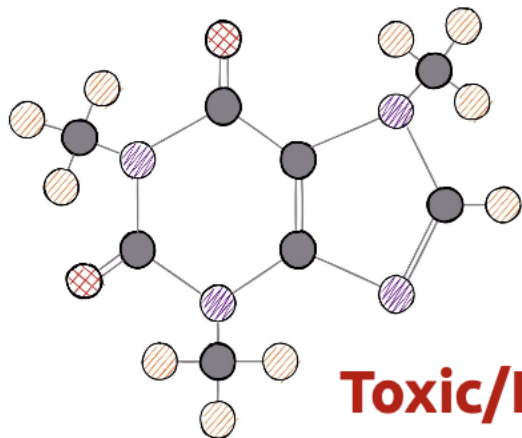
Embedding after all rounds of message passing

W and B are learnable parameters!

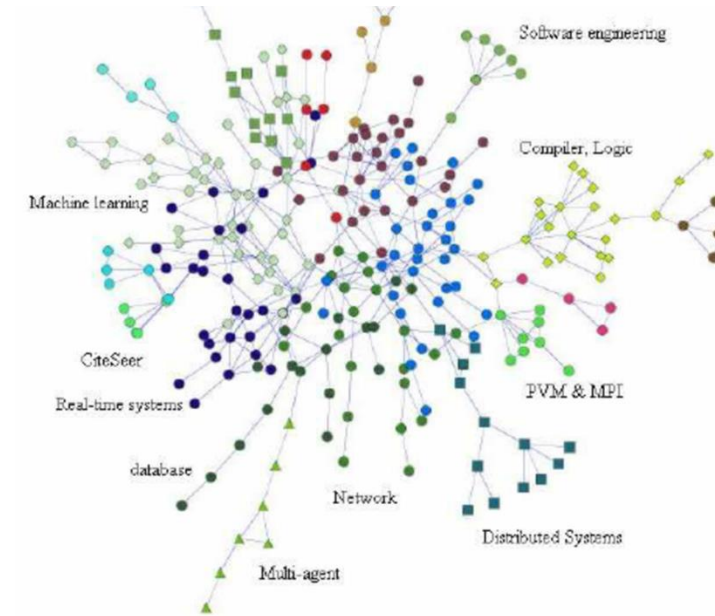
After K iterations, we end with a vector of features z_v for each node in our graph.

What might we actually want to predict?

- Node level predictions: What is the topic of a paper?
- Link Prediction: For a new paper, what other papers might it cite?
- Graph level predictions



Toxic/Not toxic?



Clustering CiteSeer collaboration network, $k = 21$. Best viewed in color.

Think/Pair/Share:

How can we turn features for each node into node-level predictions?

What about link prediction?

Graph level predictions are hard, we have $|V|$ vectors of features, which may vary between different graphs we take as input), how can we get a single output from variable sized input?

Node Prediction

Each node has a learned representation z_v , we can learn a fully-connected layer to go from features z_v to output

$$f(z_v) = \sigma(Wz_v)$$

Link Prediction

Learn a function that takes two nodes as input and predicts the presence (or absence) of an edge

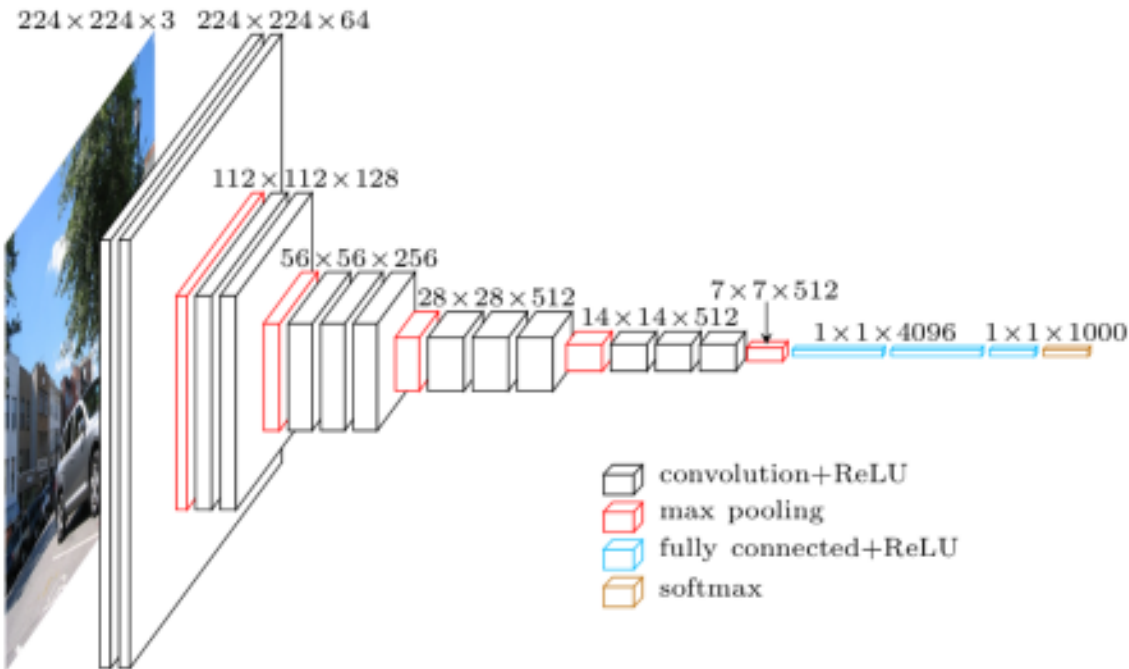
$$f(z_v, z_u) = \sigma(W(z_v \odot z_u))$$



Element-wise product

Global Pooling

- We covered MaxPool in CNNs, which looks at the maximum value in a local neighborhood
- We can consider an alternative way to do MaxPool, which looks at the entire feature for all inputs and returns a single max value

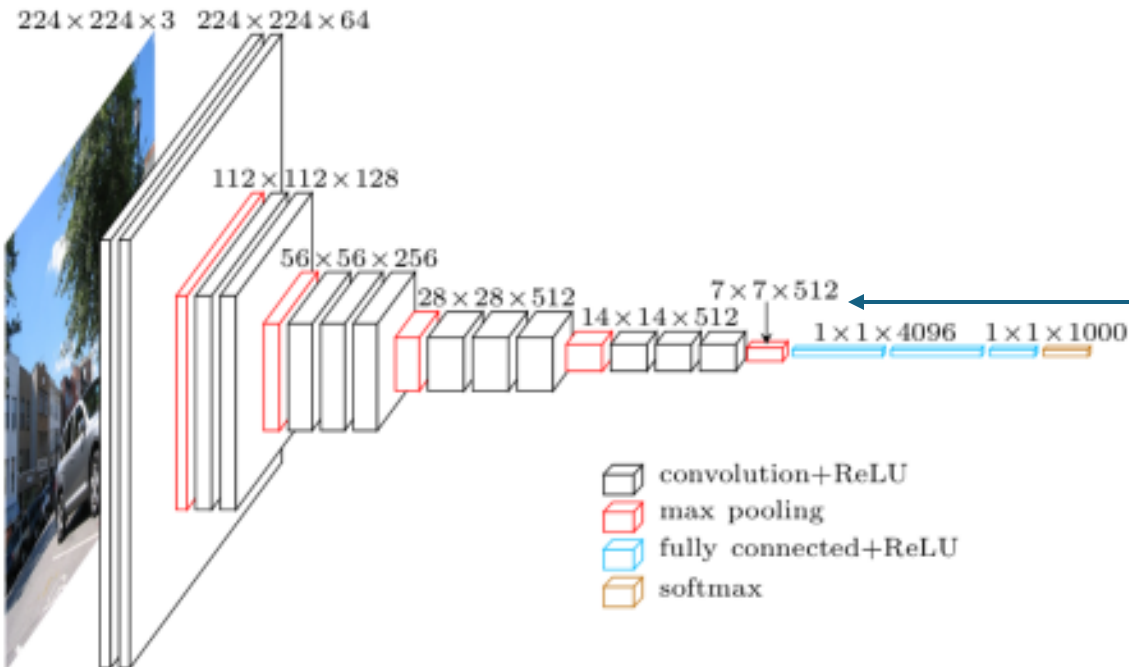


Normally, CNNs flatten the results of their final convolution and feed into a linear layer of $W \times H \times C$.

This has to be the same size for all examples!

Global Pooling

- We covered MaxPool in CNNs, which looks at the maximum value in a local neighborhood
- We can consider an alternative way to do MaxPool, which looks at the entire feature for all inputs and returns a single max value

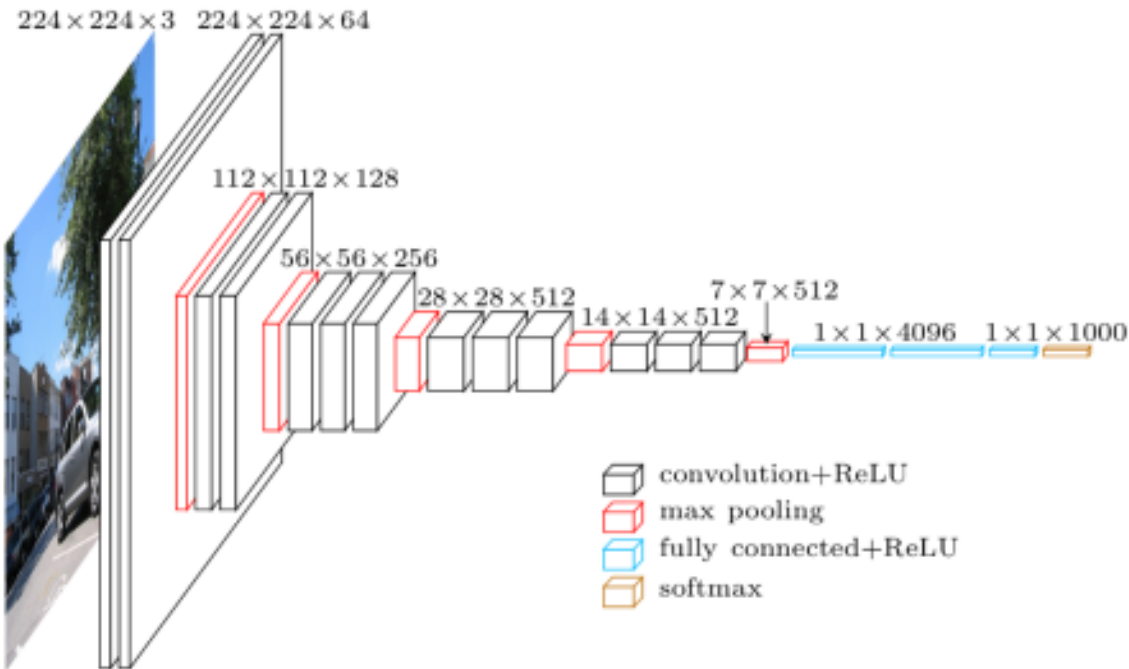


Alternatively, we can do the max over each channel in the final convolutional output

Output will have 512 features instead of 4096

Global Pooling

- We covered MaxPool in CNNs, which looks at the maximum value in a local neighborhood
- We can consider an alternative way to do MaxPool, which looks at the entire feature for all inputs and returns a single max value



MaxPooling loses lots of data from each channel... AveragePool becomes more common when performing global pooling operations

Implementing GNNs

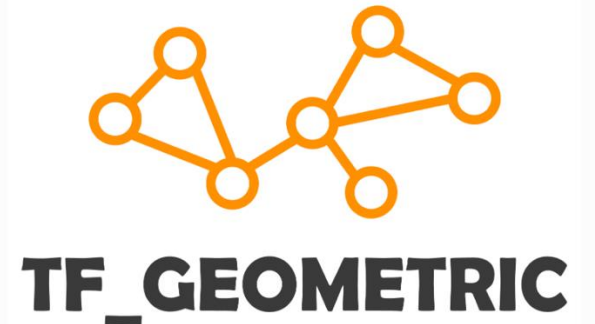
Best Resources: PyG

- Pytorch Geometric library
- Lots of industry users and well supported models and datasets

Tensorflow Version: tf_geometric

- Not as many users, but if you really like tensorflow, maybe it's for you

`SAGEConv(in_channels, out_channels)`



AlphaFold

- Goal: Given amino acid sequence, predict folding structure of protein
- Encodes each amino acid into a node and connects adjacent amino acids
- Alpha Fold 1 uses convolutions, Alpha Fold 2 uses attention (coming soon to a lecture near you)

